



**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering**

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.4.1

Binary Actuator Control with Signal Conditioning

on Embedded Systems

Checked by:

Martîniuc Alexei, *univ. asist.*
FCIM, UTM

Chişinău – 2026

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of this laboratory work is to design and implement a binary actuator control system on the Arduino Mega 2560 microcontroller that accepts commands from the user via a serial terminal, applies a multi-stage signal conditioning pipeline to eliminate spurious transitions, and drives a physical output while continuously reporting the system state on both an LCD display and the serial monitor.

The core learning objectives are to understand the concepts of binary actuator control, software-based signal conditioning for discrete signals (saturation, debouncing, and persistent state validation), and preemptive task scheduling with FreeRTOS. The lab also reinforces the principle of separating concerns across modular software layers, from raw input parsing through signal processing to hardware actuation and reporting.

1.2. Problem Definition

The task, as specified by the laboratory instructor, is reproduced below:

Control Actuator Binar

Pentru un actuator binar (ex.: bec de iluminare prin releu sau alt actuator convenit cu mentorul), se va realiza o aplicație pentru MCU care:

- *interpretează comenzi de control de la utilizator prin STDIO (serial sau keypad)*
- *controlează actuatorul*
- *afișează starea curentă pe un display LCD*

Aplicația va implementa:

- **Control actuator binar:** Primește comenzi ON/OFF prin STDIO. Aplică debouncing. Controlează starea la o recurență configurabilă (50–100 ms). Expune starea printr-o interfață internă.
- **Condiționare semnal:** Aplică saturare, debouncing software, și validare persistentă a stării.
- **Afișare și raportare:** Afișează pe LCD sau STDIO un raport structurat la 500 ms.

The system therefore requires three distinct processing stages. The user sends the string ON or OFF via the serial terminal. That raw command passes through a conditioning pipeline that first saturates the value to a strict boolean, then requires the value to remain stable for a minimum debounce window, and finally confirms it with a minimum number of consecutive identical samples before the hardware output is actually toggled. The conditioned state is displayed on the LCD and reported periodically on the serial monitor.

1.3. Used Technologies

UART Serial Communication and STDIO Redirection

UART (Universal Asynchronous Receiver-Transmitter) is the hardware serial interface used to connect the Arduino Mega 2560 to the host PC. The microcontroller's UART0 peripheral is connected to a USB-to-serial converter, making it accessible as a virtual COM port on the host. The application configures the interface at 9600 baud with the standard 8N1 framing (8 data bits, no parity, 1 stop bit).

Rather than using the Arduino-specific `Serial.print()` API at the application level, the `StdioSerial` library redirects the C standard I/O streams — `stdout` and `stdin` — to the hardware serial port using the AVR libc `fdev_setup_stream()` mechanism. This allows all formatted output to use the portable `printf()` function and all input to be read through `Serial.available()` / `Serial.read()`, keeping the application layer independent of hardware-specific serial APIs.

Binary Actuator Control and Signal Conditioning

A binary actuator is any device that operates in exactly two discrete states: active (ON) and inactive (OFF). Common examples include relays driving mains loads, solenoid valves, indicator LEDs, and buzzers. In this laboratory the actuator is simulated by a red LED connected to a current-limiting resistor on pin D7. The LED is chosen as a safe, visible proxy for a relay — the software interface is identical regardless of the actual load, since the `Led` driver abstracts the hardware behind `turnOn()`/`turnOff()` calls that could be replaced with relay driver logic without modifying any application code.

Control of a binary actuator from a user interface introduces the risk of spurious state changes caused by rapid repeated commands, communication noise, or accidental double-presses. In an industrial context, rapid cycling of relay contacts (known as "chattering") dramatically reduces the relay's mechanical lifetime and can cause arcing across the contacts, potentially damaging downstream equipment. Even in low-power applications, unnecessary state transitions waste energy and generate electromagnetic interference. A signal conditioning pipeline addresses these risks by introducing three filtering stages applied in sequence.

Stage 1 — Saturation. The first stage maps any raw input value to a strict element of the boolean set `{false, true}`. While this is trivially satisfied for a serial command parser that already outputs a boolean, saturation becomes essential when the input source is an analog sensor, a noisy digital line, or a multi-valued protocol message. By placing saturation at the pipeline entry, the subsequent stages are guaranteed to operate on clean boolean values regardless of the input source.

Stage 2 — Software Debouncing. The second stage prevents the actuator state from changing unless the input has been stable for a minimum duration (configured as 100 ms in this laboratory). This mirrors the hardware debouncing technique used for mechanical pushbuttons and relay contacts, where the physical switch bounces between open and closed states for several milliseconds

after activation. Software debouncing uses a timer that resets every time the input changes; only when the timer exceeds the debounce threshold is the input considered stable. The debounce period must be chosen to exceed the longest expected transient duration while remaining short enough to preserve acceptable response latency.

Stage 3 — Persistent State Validation. The third stage requires the input to be confirmed by a minimum number of consecutive identical samples (configured as 3 confirmations in this laboratory). This provides an additional layer of filtering beyond time-based debouncing: even if a transient glitch is long enough to survive the debounce window, it must appear consistently across multiple sampling intervals to be accepted. The dual-guard design — requiring *both* the debounce period and the persistence count to be satisfied simultaneously — ensures robust noise rejection suitable for safety-critical applications as described in IEC 61508.

FreeRTOS Preemptive Task Scheduler

FreeRTOS is a real-time operating system kernel that provides preemptive multitasking on resource-constrained microcontrollers. The kernel maintains a ready queue of tasks sorted by priority, and the scheduler always dispatches the highest-priority ready task. When a higher-priority task becomes ready (for example, because its delay period has elapsed), the kernel immediately preempts the currently running task, saves its context (register file and stack pointer) to the task control block (TCB), and restores the context of the higher-priority task. This preemption happens transparently via a timer interrupt — on the AVR ATmega2560, the kernel uses Timer 3 configured at a default tick rate of 200 Hz (5 ms per tick).

On the Arduino Mega 2560, the `feilipu/FreeRTOS` library ports the kernel to the AVR architecture. Because the ATmega2560 has only 8 KB of SRAM and each task requires its own stack, careful stack sizing is critical. The `portSTACK_TYPE` on AVR is `uint8_t`, meaning stack sizes specified in the `xTaskCreate()` call are in bytes, not words. A task that uses `printf()` requires at least 200–300 bytes of stack space for the formatting buffer alone, plus additional space for local variables, function call frames, and the saved register context (32 general-purpose registers plus SREG and the program counter on AVR). In this laboratory, the input task is allocated 256 bytes, the actuator task 256 bytes, and the display task 512 bytes (to accommodate the larger `printf()` formatting calls in the periodic STDIO report).

Each task runs a while-forever loop and uses `vTaskDelayUntil()` rather than `vTaskDelay()` for period control. The difference is significant: `vTaskDelay()` delays for a specified number of ticks *from the moment the call is made*, which means the actual period includes the task's execution time and varies with processing load. In contrast, `vTaskDelayUntil()` delays until an absolute tick count is reached, compensating for the execution time and producing a consistent period regardless of how long the task body takes to execute. This is essential for the actuator control task, which must run at a predictable 75 ms recurrence to satisfy the conditioning pipeline's timing assumptions.

Preemptive scheduling ensures that higher-priority tasks (input reading at priority 3) can

interrupt lower-priority ones (display at priority 1), guaranteeing responsive user interaction regardless of display rendering time.

Mutex-Based Synchronization. When multiple tasks share data, a synchronization mechanism is required to prevent race conditions — situations where one task reads a partially-updated structure written by another. FreeRTOS provides binary semaphores and mutexes for this purpose. A mutex (mutual exclusion semaphore) additionally implements priority inheritance: if a low-priority task holds the mutex and a high-priority task attempts to take it, the kernel temporarily raises the holder's priority to match the waiter's, preventing unbounded priority inversion. In this laboratory, the `ActuatorShared` structure is protected by a single FreeRTOS mutex. Each task acquires the mutex with a 5 ms timeout, performs its read or write operation, and immediately releases it. The critical sections are kept deliberately short — typically a single field assignment — to minimize the time any task blocks waiting for access.

I2C Protocol and LCD Display

The Inter-Integrated Circuit (I2C) protocol is a two-wire synchronous serial bus originally developed by Philips Semiconductor (now NXP) for short-distance intra-board communication. The bus uses two open-drain signal lines — SDA (Serial Data) and SCL (Serial Clock) — both pulled high by external resistors. Communication follows a master-slave architecture: the master (the Arduino Mega 2560 in this case) generates the clock and initiates transactions, while the slave (the PCF8574 I/O expander on the LCD backpack) responds with acknowledgements and data.

Each I2C transaction begins with a START condition (SDA falling while SCL is high), followed by a 7-bit slave address and a read/write direction bit. The addressed slave acknowledges by pulling SDA low during the ninth clock cycle. Data bytes follow, each acknowledged individually, until the master issues a STOP condition (SDA rising while SCL is high). The Arduino Mega 2560 provides a dedicated hardware I2C peripheral (TWI — Two-Wire Interface) on pins SDA (D20) and SCL (D21), which handles the low-level bit timing and acknowledgement detection in hardware, freeing the CPU for other tasks.

The LCD module used in this laboratory is a 16×2 character display based on the Hitachi HD44780 controller. The HD44780 natively requires an 8-bit or 4-bit parallel data interface plus three control signals (RS, RW, E), which would consume 7–11 GPIO pins on the microcontroller. The PCF8574 I2C backpack solves this by converting the two-wire I2C bus into the parallel interface required by the HD44780, reducing the pin count to just SDA and SCL. The standard I2C address of 0x27 is used for the LCD module; this address is set by the A0–A2 address pins on the PCF8574, which are all pulled high by default.

The `LcdDisplay` library wraps the third-party `LiquidCrystal_I2C` driver, providing a simplified line-based API. The `showTwoLines()` method writes both LCD lines in a single call, automatically padding shorter strings with spaces to clear residual characters from the previous update. This prevents visual artifacts where old characters remain visible when the new string is shorter than the previous one — a common issue when directly using the `setCursor()/print()`

API without explicit clearing.

1.4. Hardware Components

Arduino Mega 2560

The Arduino Mega 2560 is a microcontroller development board built around the 8-bit AVR ATmega2560 running at 16 MHz. It provides 256 KB of Flash program memory, 8 KB of SRAM for runtime data, 54 digital I/O pins, and 16 analog inputs. Four hardware UART ports enable simultaneous serial communication with the host PC and other peripherals. The on-board USB-to-serial converter allows programming and monitoring over a standard USB cable. In this laboratory the board acts as the system's central processing unit, executing the FreeRTOS kernel and all three application tasks.

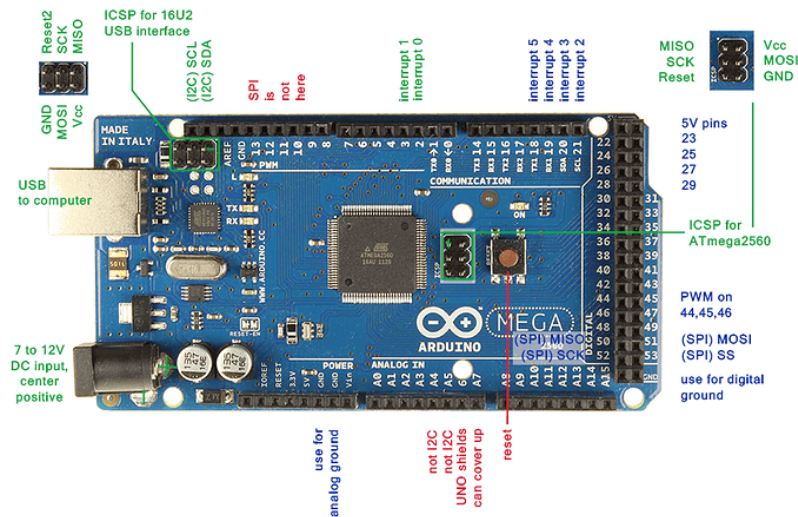


Figure 1.1 – Arduino Mega 2560 development board

LED and Current-Limiting Resistor

A standard 5 mm red LED is used to simulate the binary actuator output. An LED requires a current-limiting resistor in series to prevent excessive current from damaging the device. With a supply of 5 V, a typical LED forward voltage of 2.0 V, and a desired current of approximately 14 mA, the required resistance is $(5.0 - 2.0)/0.014 \approx 214\Omega$, so a standard 220 Ω resistor is used. In a real-world application the LED and resistor would be replaced by a relay module driving a mains load; the software interface and signal conditioning pipeline remain identical regardless of the actual load type.

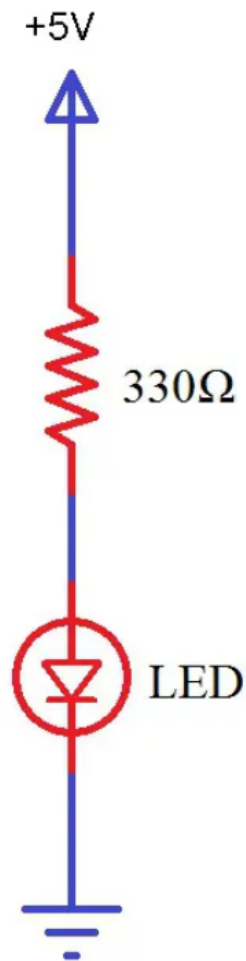


Figure 1.2 – 5 mm red LED (simulates binary actuator output)

1.5. Software Components

PlatformIO

PlatformIO is an open-source embedded development ecosystem integrated into VS Code. It manages board configurations, toolchain installation, library dependency resolution, and project building. Each lab is defined as a named environment in `platformio.ini` with its own build flags, source filter, and library list. Switching between labs requires only selecting a different environment in the IDE — no source files need to be modified. The `pio run -e lab4_1` command compiles the firmware for this lab.

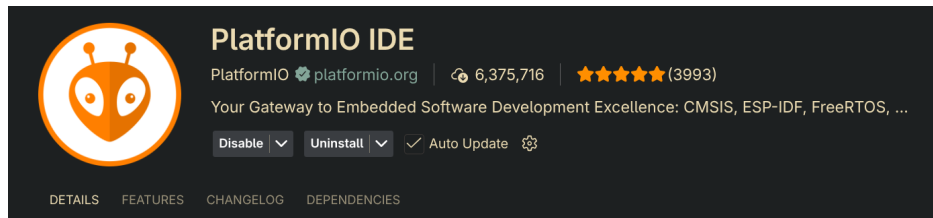


Figure 1.3 – PlatformIO IDE in VS Code with Lab 4.1 environment

Wokwi

Wokwi is a browser-based and VS Code-integrated simulator for embedded systems that supports the Arduino Mega 2560. It loads the compiled PlatformIO firmware directly from `.pio/build/lab4`, simulating both the microcontroller execution and the virtual serial terminal. This allows functional testing — sending ON/OFF commands and observing the LED and LCD behaviour — without physical hardware.

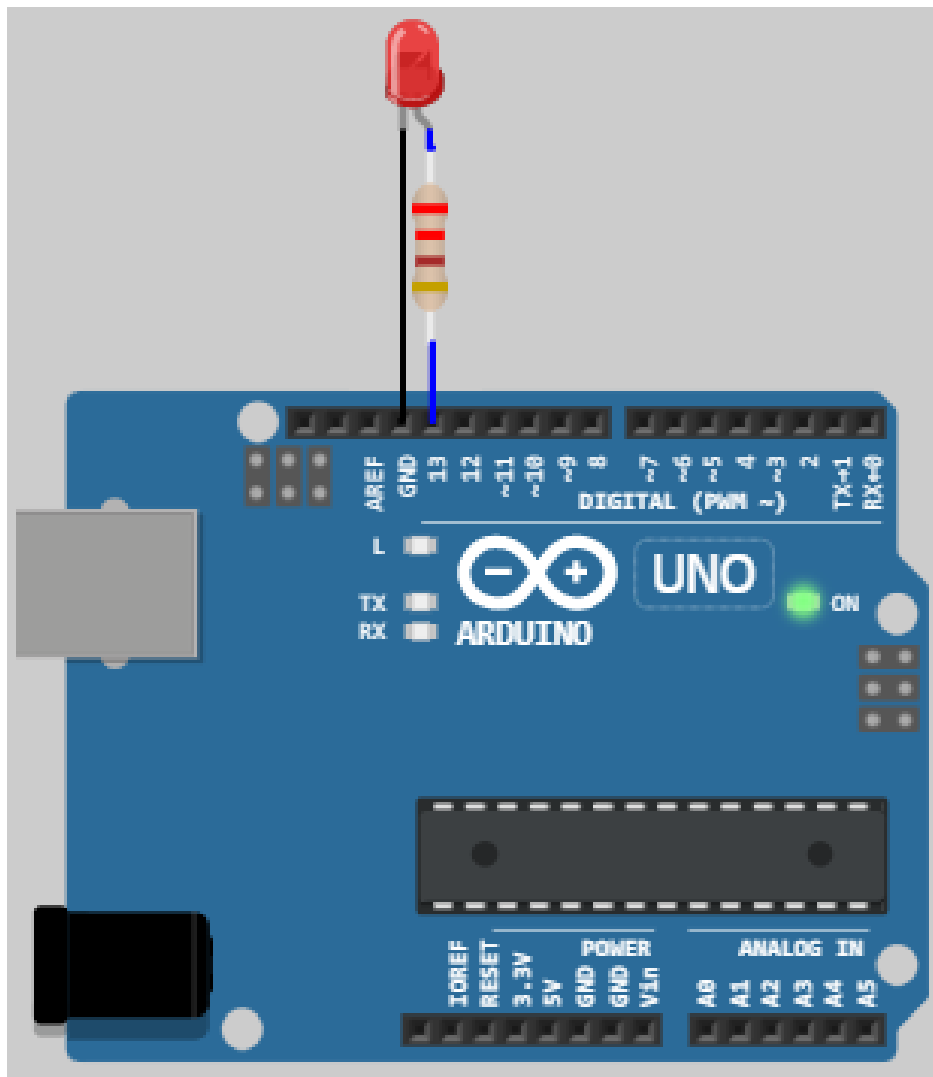


Figure 1.4 – Wokwi simulator running the Lab 4.1 circuit

1.6. System Architecture and Justification

The system follows a layered hardware-abstraction architecture divided into five tiers. The hardware tier consists of the physical GPIO pin, resistor, and LED. The MCAL (Microcontroller Abstraction Layer) tier is represented by the Led driver, which encapsulates all `pinMode()` and `digitalWrite()` calls behind a stable API. The ECAL (ECU Abstraction Layer) tier contains the `BinaryConditioner`, which implements the signal processing logic independent of any specific hardware. The service tier is represented by `ActuatorControl`, which combines the conditioner with the LED driver to form a self-contained actuator module. The application tier consists of the three FreeRTOS tasks and the shared state structure that orchestrate the overall system behaviour.

FreeRTOS is chosen over a simple cooperative scheduler because it provides true preemption and priority enforcement — the input reader task at priority 3 will always preempt the display task at priority 1, ensuring that serial input is never missed regardless of how long the display update takes. The mutex on the shared state structure prevents partially-written structs from being read by another task mid-update.

1.7. Case Study — Relevant Applicability of the Solution

Binary actuator control with signal conditioning is ubiquitous in industrial automation. A programmable logic controller (PLC) controlling a conveyor-belt drive motor, a solenoid valve in a pneumatic system, or a contactor in a motor control centre all employ debounce and persistence logic to prevent spurious state transitions from noisy digital inputs or momentary command spikes. The same pattern appears in building management systems where light relays and HVAC damper actuators must not cycle rapidly in response to transient occupancy sensor readings, since relay contact wear increases with switching frequency.

In consumer electronics, smart home relay modules use software debouncing to prevent a user's voice command from being applied multiple times due to repeated speech recognition events. In automotive body-control modules, door-lock actuators are controlled through validated discrete inputs to prevent uncommanded locking caused by electromagnetic interference on vehicle wiring harnesses. The laboratory's three-stage pipeline — saturation, debounce, persistence — directly mirrors the input processing found in safety-critical embedded systems described by the ISO 26262 and IEC 61508 functional-safety standards.

2. Design

2.1. System Architecture Diagrams

Component-Level Architecture

The system is composed of four hardware-connected components and one host interface, all orchestrated by the Arduino Mega 2560 microcontroller. The serial terminal running on the host PC provides the command-input path over the USB-to-UART bridge; the red LED on digital

pin D7 acts as the binary actuator output (simulating a relay or solenoid); and the 16x2 I2C LCD provides continuous visual feedback. All components share a common ground and the 5 V supply provided by the board's voltage regulator. The bidirectional data flow between the PC and the MCU is achieved through UART0 at 9600 baud: the user types ON or OFF commands that travel from the terminal to the MCU, while status confirmation messages and periodic structured reports travel from the MCU back to the terminal.

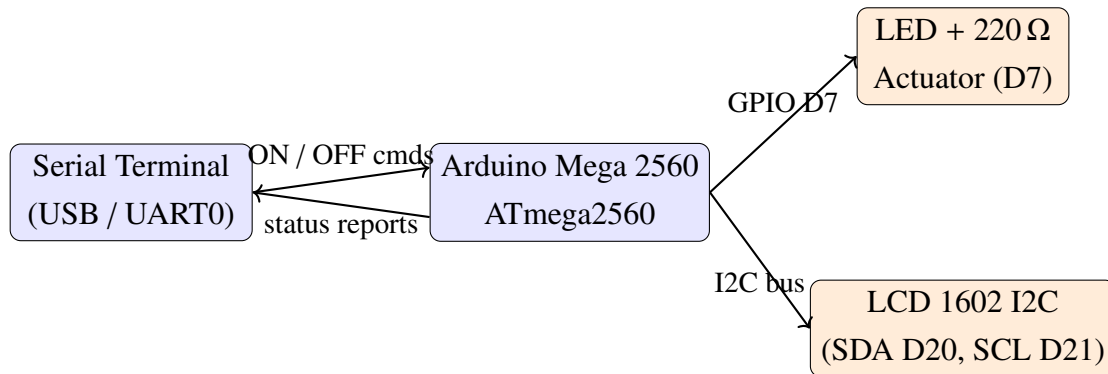


Figure 2.1 – System component-level architecture for Lab 4.1

The component diagram shows that the Arduino Mega 2560 is the central hub of the system. The serial terminal is the sole input source: the user types a command and the microcontroller's UART0 receive register captures the bytes. After parsing and conditioning, the output path branches into two: the GPIO output toggles the actuator LED, and the I2C output updates the LCD display. This simple fan-out topology is characteristic of small embedded control systems where a single processor manages all peripherals without external bus arbitration or multi-master contention.

Layered System Architecture

The software is organized into five layers following the automotive-inspired AUTOSAR-like layered architecture used consistently throughout the lab series. Each layer communicates exclusively with its immediate neighbours through well-defined header-file interfaces. This strict layering ensures that changes to one layer (for example, replacing the LED with a relay module) propagate only to the layer directly above or below.

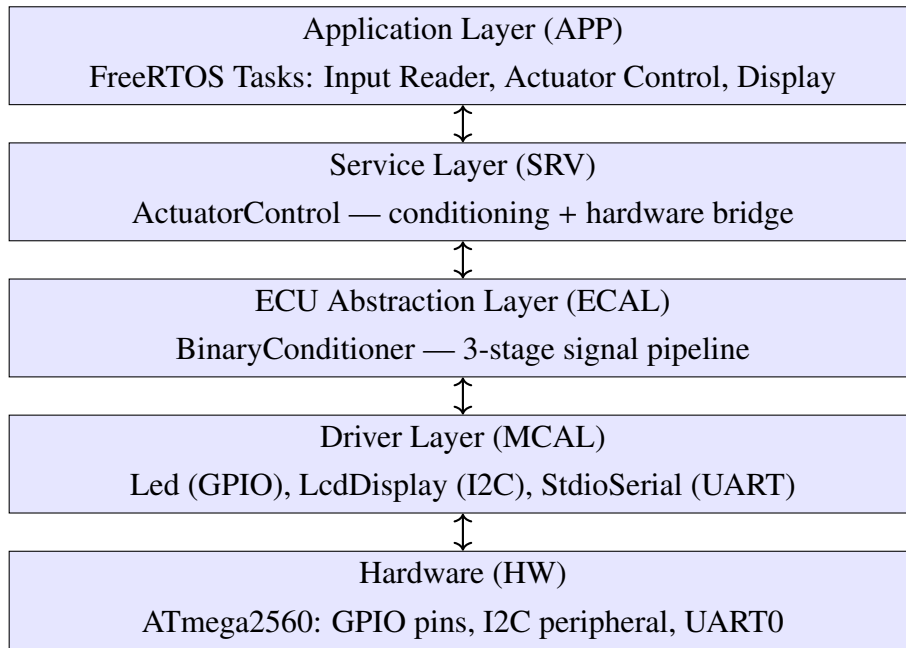


Figure 2.2 – Layered software architecture of the Lab 4.1 system

The **Hardware (HW)** layer consists of the physical ATmega2560 microcontroller with its GPIO peripheral (pin D7 driving the actuator LED), the TWI/I2C peripheral (pins D20/D21 driving the LCD backpack), and the UART0 peripheral (connected to USB for host communication).

The **Driver Layer (MCAL)** contains three reusable library modules. The `Led` class encapsulates all `pinMode()` and `digitalWrite()` calls behind a clean API of `init()`, `turnOn()`, `turnOff()`, `toggle()`, and `isOn()` methods. The `LcdDisplay` class wraps the `LiquidCrystal_I2C` library into a line-based display interface with automatic padding. The `StdioSerial` module redirects the C standard I/O streams to UART0 using `fdev_setup_stream()`.

The **ECU Abstraction Layer (ECAL)** contains the `BinaryConditioner` class, which implements the three-stage signal conditioning pipeline (saturation, debouncing, persistent validation) in a purely software domain, operating entirely on boolean values with no hardware knowledge.

The **Service Layer (SRV)** contains `ActuatorControl`, which wires the `BinaryConditioner` to the `Led` driver. It accepts raw ON/OFF commands, processes them through the conditioner, and updates the hardware only when the conditioned state changes.

The **Application Layer (APP)** consists of the three FreeRTOS tasks and the shared state structure (`ActuatorShared`) that orchestrate the overall system behaviour.

2.2. Block Diagrams

System Data Flow and FreeRTOS Task Architecture

The three FreeRTOS tasks communicate through a mutex-protected shared structure `g_actuator`. The diagram below illustrates the data-flow direction between tasks and the shared memory, as well as the period and priority of each task. Task 1 (Input Reader) is the highest-priority task because it must respond to serial input within one polling interval (10 ms). Task 2 (Actuator Control) runs at

medium priority to process the conditioning pipeline every 75 ms. Task 3 (Display/Report) runs at the lowest priority since display updates are the least time-sensitive operation.

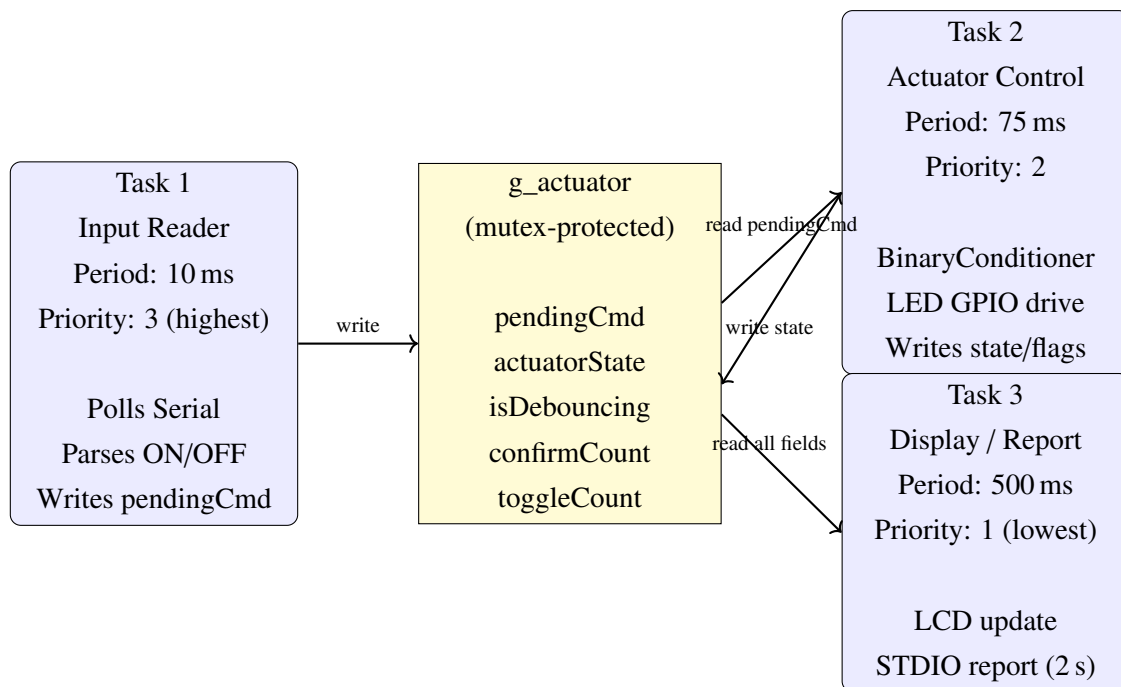


Figure 2.3 – FreeRTOS task data-flow diagram for Lab 4.1

The shared structure uses a FreeRTOS mutex (`xSemaphoreCreateMutex()`) to prevent data races. Every read or write to the shared fields is bracketed by `xSemaphoreTake()` and `xSemaphoreGive()` calls with a 5–10 ms timeout. The short timeout ensures that if one task is delayed, the others are not blocked indefinitely. In practice, the critical sections are short (a few microseconds for reading/writing primitive fields), so mutex contention is negligible.

Input Reader Task Flowchart

The input reader task polls the hardware serial port at 10 ms intervals. Incoming characters are accumulated into a line buffer with local echo. When a newline is received, the buffer is trimmed and upcased, then matched against the known commands ON and OFF. Valid commands are written to `g_actuator.pendingCmd` under mutex protection; unrecognized input is rejected with an error message.

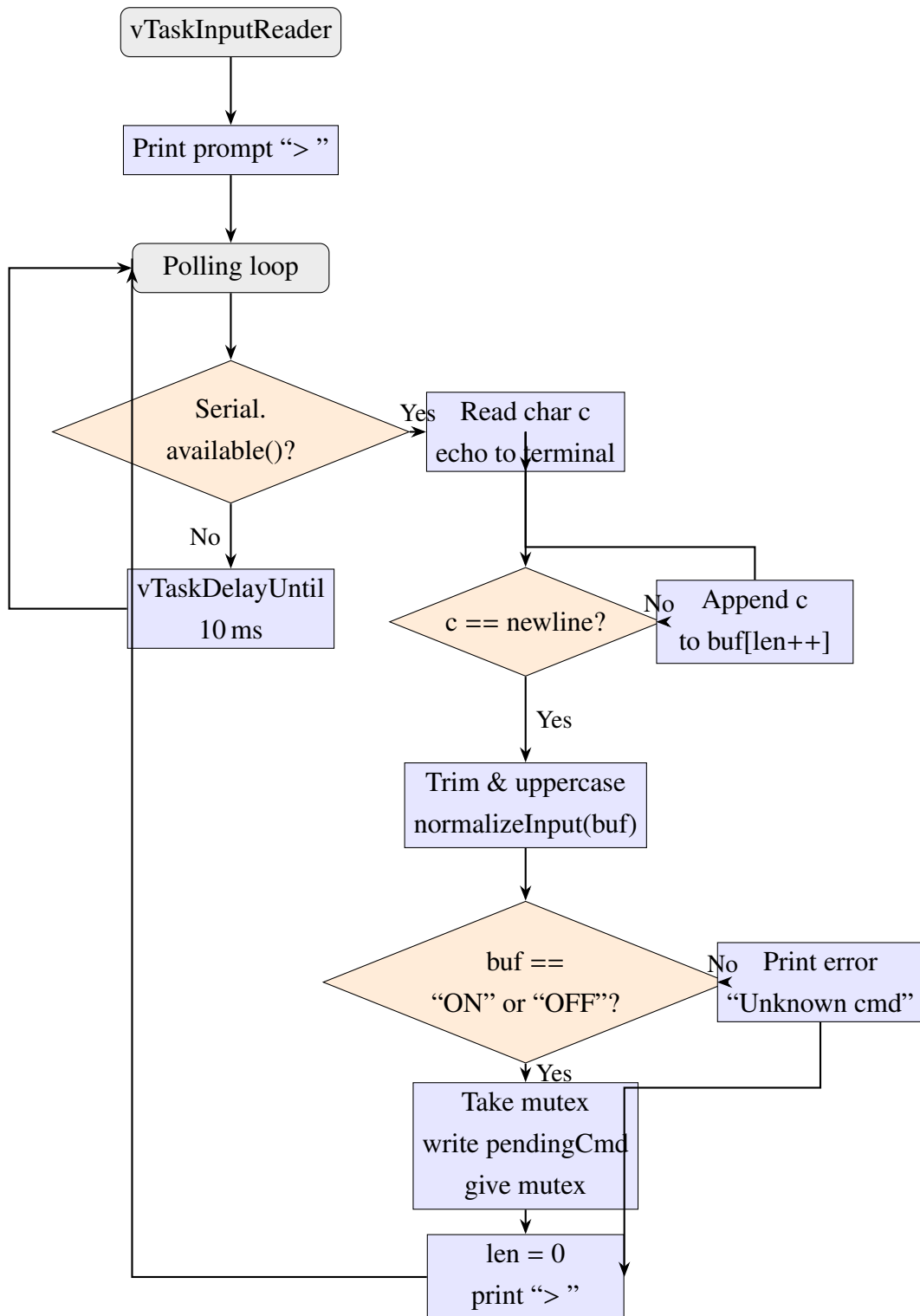


Figure 2.4 – Flowchart of the Input Reader task (*vTaskInputReader*)

BinaryConditioner Pipeline Flowchart

The `BinaryConditioner::process()` function is the heart of the signal conditioning pipeline. It is called once per Actuator Control task tick (every 75 ms) with the current raw command. The three stages — saturation, debouncing, and persistent validation — are evaluated in sequence. A state transition is committed to the conditioned output only when *both* the debounce timer has elapsed *and* the required number of consecutive identical samples has been reached.

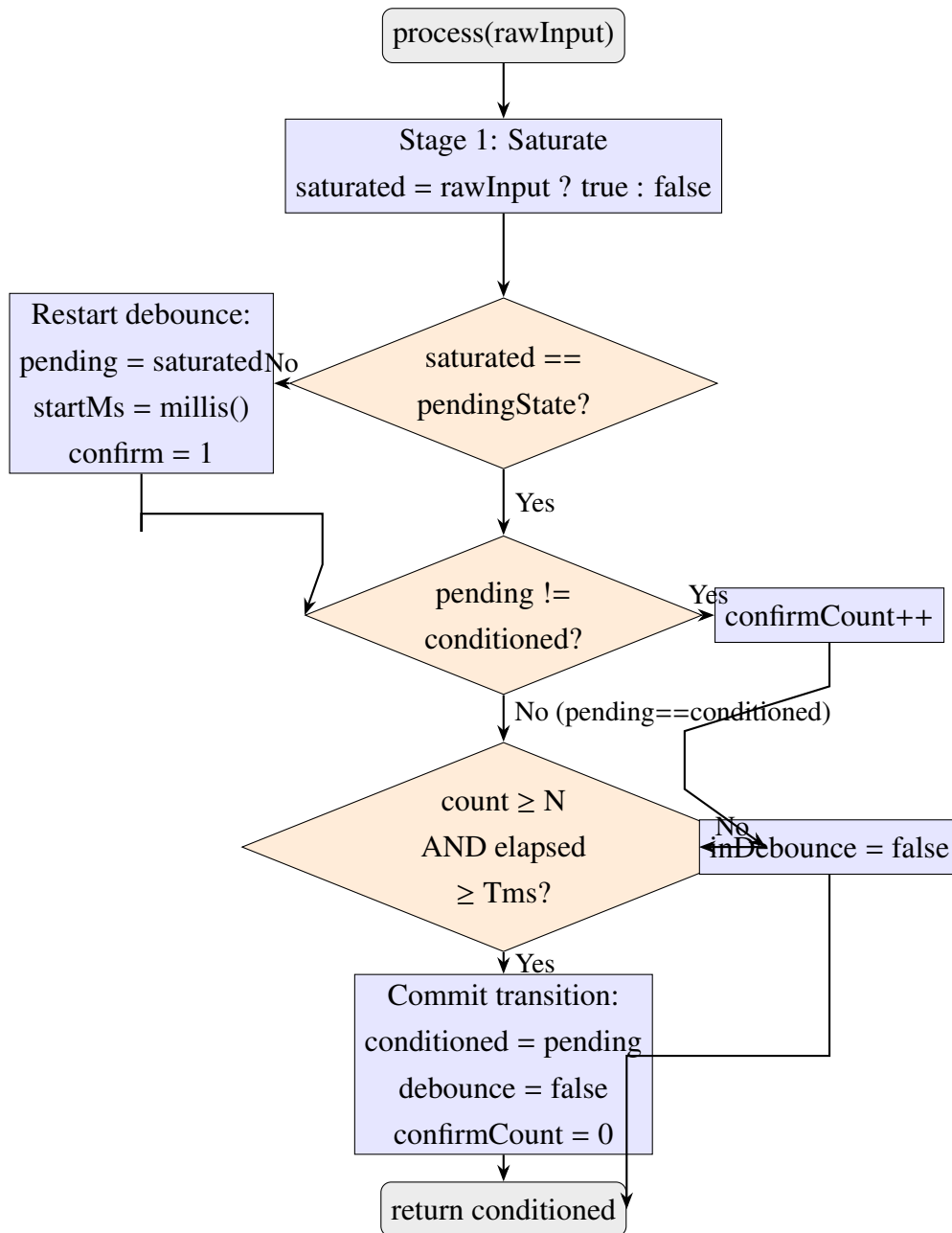


Figure 2.5 – Flowchart of *BinaryConditioner::process()* — 3-stage signal conditioning

The critical design decision visible in this flowchart is the dual guard on state commitment: both the debounce timer (Stage 2) and the persistence counter (Stage 3) must be satisfied simultaneously. If the input oscillates rapidly, the persistence counter resets before reaching N , blocking the transition even if the debounce window has elapsed. Conversely, if the input stabilizes instantly after a change but then reverts before the debounce window expires, the transition is still blocked. This makes the conditioner robust against both periodic glitches and burst noise.

Actuator Control Task Flowchart

The Actuator Control task runs at 75 ms. On each tick it reads the pending command from the shared state, passes it through the `ActuatorControl::update()` method (which internally invokes `BinaryConditioner::process()` and drives the LED), and writes the resulting condi-

tioned state back to the shared memory.

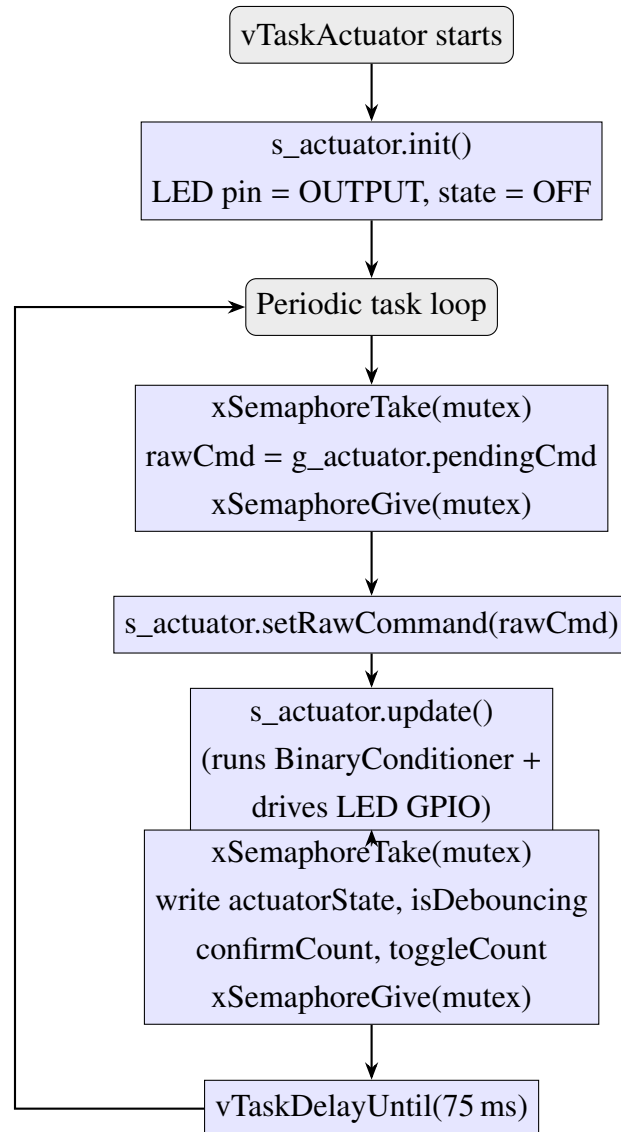


Figure 2.6 – Flowchart of the Actuator Control task (`vTaskActuator`)

Display and Reporting Task Flowchart

The Display task runs at 500 ms. It reads the complete shared state under mutex protection, formats and displays the actuator status on the I2C LCD, and every four ticks (approximately every 2 seconds) prints a structured report to the serial terminal via `printf()`.

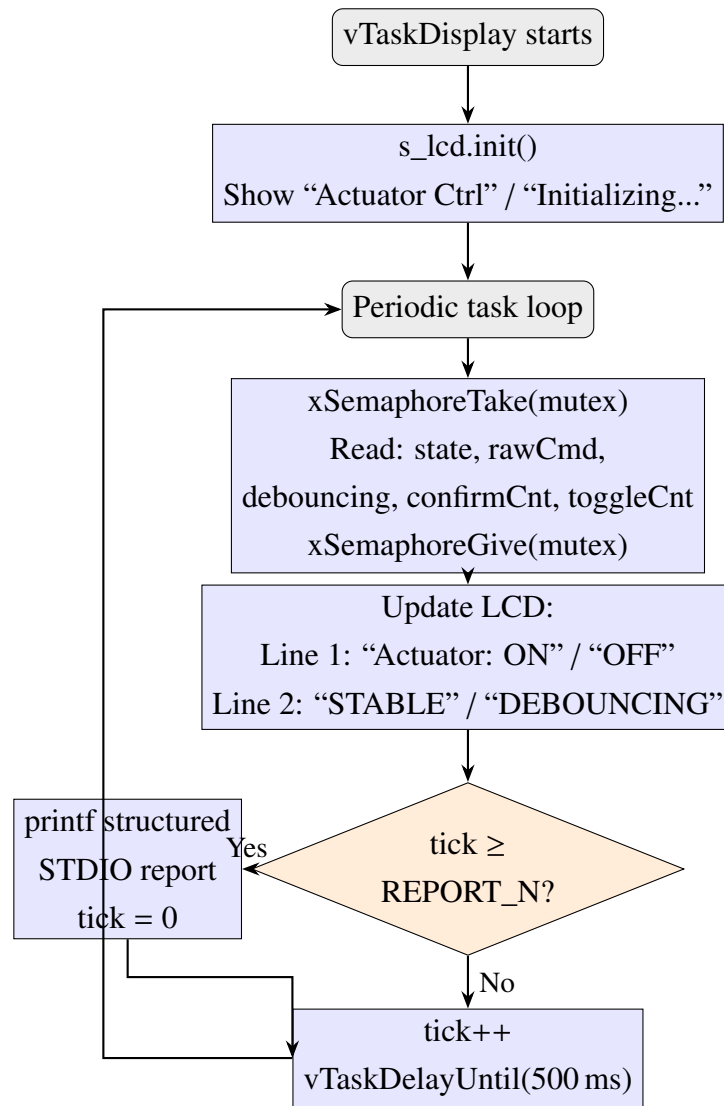


Figure 2.7 – Flowchart of the Display and Reporting task (*vTaskDisplay*)

ActuatorControl::update() Flowchart

The `update()` method in the `ActuatorControl` service layer converts the raw command to a boolean, passes it through the `BinaryConditioner::process()` pipeline, and conditionally updates the hardware LED. The GPIO output is only toggled when the conditioned state actually changes, avoiding unnecessary `digitalWrite()` calls on every tick.

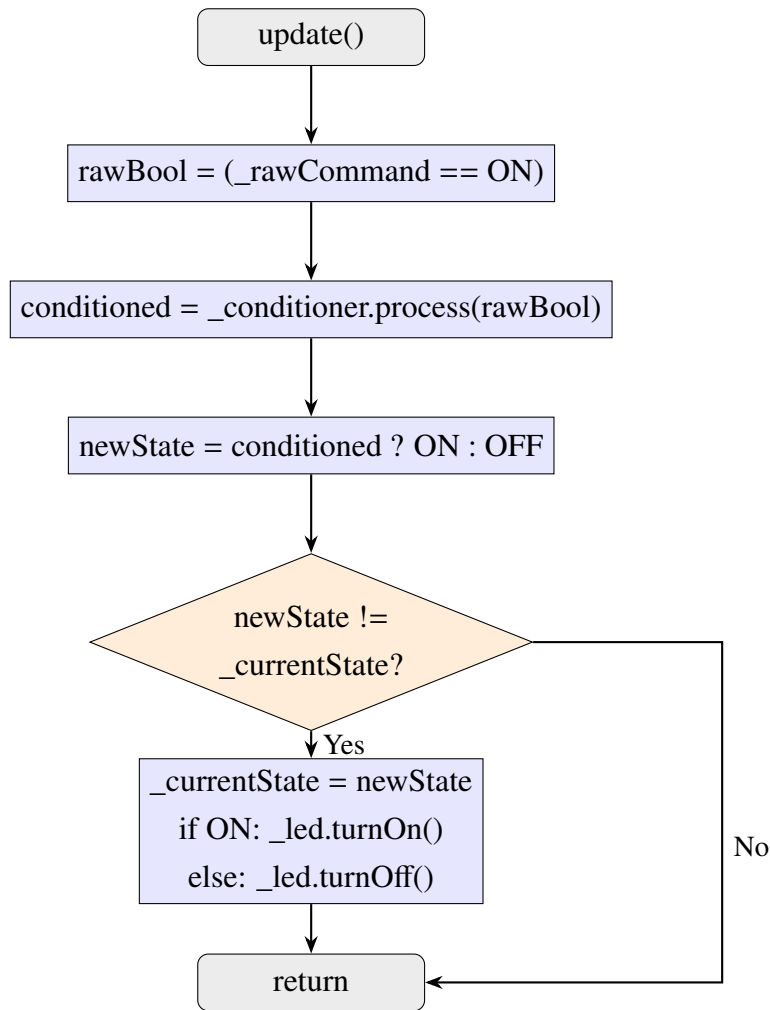


Figure 2.8 – Flowchart of `ActuatorControl::update()`

2.3. Electrical Schematics

Component Specification

The circuit uses three components beyond the Arduino Mega 2560 development board itself:

1. **Red LED (5 mm):** The binary actuator output on pin D7. Forward voltage: approximately 2.0 V. Maximum forward current: 20 mA. The LED simulates a real-world binary actuator such as a relay coil or solenoid driver.
2. **220 Ω Resistor:** Connected in series with the LED to limit current. With a 5 V supply and a 2.0 V LED forward voltage, the current is $I = (5.0 - 2.0)/220 \approx 13.6$ mA, which is safely below the 20 mA maximum and the ATmega2560's 20 mA per-pin GPIO current limit.
3. **LCD 1602 I2C Module:** A 16-column, 2-row character LCD with a PCF8574 I2C backpack. The module is powered from the 5 V rail and communicates over the I2C bus using the standard 7-bit address 0x27. The SDA and SCL lines are connected to the Mega's dedicated I2C pins (D20 and D21 respectively). Internal pull-up resistors on the backpack module eliminate the need for external I2C pull-ups.

Circuit Connections

The electrical connections are illustrated in the schematic below. The LED branch forms a simple series path: pin D7 sources current through the $220\ \Omega$ resistor and the LED to ground. The I2C LCD requires four connections: VCC (5 V), GND, SDA (D20), and SCL (D21).

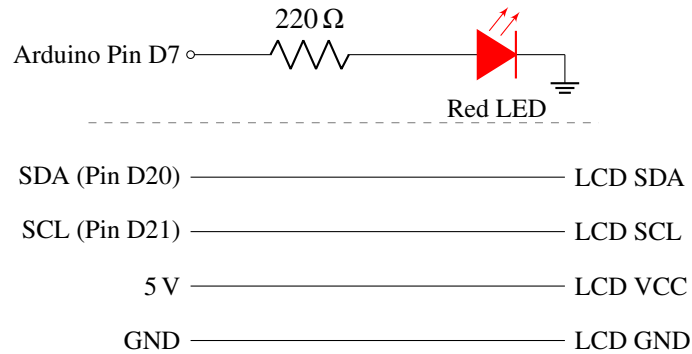


Figure 2.9 – Electrical schematic for Lab 4.1 — actuator LED circuit and I2C LCD connections

Pin Mapping Summary

Table 1: Pin mapping for Lab 4.1

Arduino Pin	Component	Function
D7	Red LED (via $220\ \Omega$)	Binary actuator output
D20 (SDA)	LCD 1602 I2C backpack	I2C data line
D21 (SCL)	LCD 1602 I2C backpack	I2C clock line
USB (UART0)	Host PC serial terminal	Command input / status output
5V	LCD module VCC	Power supply
GND	LED cathode, LCD GND	Common ground

Hardware Configuration

The complete Wokwi simulation is defined in `labs/wokwi/lab4.1/diagram.json` and loaded via `wokwi.toml`. The configuration below lists all virtual components and their wire connections.

Listing 2.1 – Wokwi hardware configuration (`diagram.json`)

```
{
  "version": 1,
  "author": "Vremere Adrian",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-arduino-mega", "id": "mega",
      "top": 0, "left": 0 },
    { "type": "wokwi-lcd1602", "id": "lcd1",
      "top": -320, "left": 100,
```

```

    "attrs": { "pins": "i2c" } },
  { "type": "wokwi-led",          "id": "led_act",
    "top": 280, "left": 220,
    "attrs": { "color": "red", "label": "Actuator" } },
  { "type": "wokwi-resistor",     "id": "r_act",
    "top": 245, "left": 200, "rotate": 90,
    "attrs": { "resistance": "220" } }
],
"connections": [
  ["lcd1:VCC", "mega:5V",    "red",
   ["v20", "h-60", "v200"]],
  ["lcd1:GND", "mega:GND.1", "black",
   ["v20", "h-40", "v200"]],
  ["lcd1:SDA", "mega:20",    "cyan",
   ["v40", "h100", "v140"]],
  ["lcd1:SCL", "mega:21",    "darkgreen",
   ["v60", "h120", "v120"]],
  ["mega:7",    "r_act:1",    "orange",
   ["v60", "h0", "v80"]],
  ["r_act:2",   "led_act:A",  "orange", ["v0"]],
  ["led_act:C", "mega:GND.2", "black",
   ["v30", "h-160", "v-200"]]
]
}

```

The `wokwi.toml` file configures the Wokwi VS Code extension to load the compiled firmware directly from the PlatformIO build output directory:

Listing 2.2 – Wokwi firmware configuration (`wokwi.toml`)

```

[wokwi]
version = 1
firmware = "../../pio/build/lab4_1/firmware.hex"
elf = "../../pio/build/lab4_1/firmware.elf"

```

[Screenshot pending — run the Wokwi simulation and save to
resources/Design/ElectricalSchematics/wokwi_circuit.png]

Figure 2.10 – Wokwi simulation circuit (active run — LED lit, LCD showing actuator state)

2.4. Project Structure

The Lab 4.1 code is organized within the shared PlatformIO project under `labs/`, following the convention established in previous labs. Each lab has its own entry-point directory and Wokwi simulation folder, while reusable library code is shared under `lib/`.

Listing 2.3 – Lab 4.1 project directory structure

```

labs/

```

```

|-- platformio.ini          # Build config (env:lab4_1)
|-- src/
|   |-- main.cpp           # Entry point (lab selector via #ifdef LAB4_1)
|-- lab/
|   |-- lab4_1/
|       |-- lab4_1_main.h   # Lab 4.1 entry point interface
|       |-- lab4_1_main.cpp # Lab 4.1 entry point (task creation)
|       |-- actuator_data.h # Shared state structure + constants
|       |-- actuator_data.cpp # Shared state initialization
|       |-- task_input.h    # Input reader task interface
|       |-- task_input.cpp  # Input reader task implementation
|       |-- task_actuator.h # Actuator control task interface
|       |-- task_actuator.cpp # Actuator control task implementation
|       |-- task_display.h  # Display/report task interface
|       |-- task_display.cpp # Display/report task implementation
|-- lib/
|   |-- ActuatorControl/    # SRV: conditioner + LED bridge
|   |   |-- ActuatorControl.h
|   |   |-- ActuatorControl.cpp
|   |-- BinaryConditioner/  # ECAL: 3-stage binary pipeline
|   |   |-- BinaryConditioner.h
|   |   |-- BinaryConditioner.cpp
|   |-- Led/                # MCAL: GPIO LED driver
|   |   |-- Led.h
|   |   |-- Led.cpp
|   |-- LcdDisplay/         # MCAL: I2C LCD driver
|   |   |-- LcdDisplay.h
|   |   |-- LcdDisplay.cpp
|   |-- StdioSerial/        # MCAL: STDIO -> UART redirection
|   |   |-- StdioSerial.h
|   |   |-- StdioSerial.cpp
|-- wokwi/
|   |-- lab4.1/
|       |-- diagram.json    # Wokwi virtual circuit definition
|       |-- wokwi.toml      # Firmware path for Wokwi loader

```

Each library module under `lib/` is a self-contained, reusable unit with a clear public interface (header file) and a hidden implementation (source file). The lab-specific orchestration code resides in `lab/lab4_1/`, while `src/main.cpp` delegates to the active lab's `setup()` and `loop()` functions via preprocessor conditionals.

2.5. Modular Implementation

MCAL Layer: Led Driver

The `Led` class provides a hardware abstraction for controlling a single LED connected to a GPIO pin. It encapsulates the Arduino-specific `pinMode()` and `digitalWrite()` calls behind a clean object-oriented interface. The internal state boolean tracks the current LED state, so callers

never need to call `digitalRead()` to query whether the LED is on or off.

Listing 2.4 – Led driver interface (lib/Led/Led.h)

```
#ifndef LED_H
#define LED_H

#include <Arduino.h>

class Led {
public:
    Led(uint8_t pin);           // Construct with GPIO pin number
    void init();               // Set OUTPUT, initial OFF
    void turnOn();             // Set HIGH, state = true
    void turnOff();            // Set LOW, state = false
    void toggle();             // Invert current state
    bool isOn() const;         // Query current state
private:
    uint8_t ledPin;            // GPIO pin number
    bool    state;             // Current state (true = ON)
};

#endif // LED_H
```

Listing 2.5 – Led driver implementation (lib/Led/Led.cpp)

```
#include "Led.h"

Led::Led(uint8_t pin) : ledPin(pin), state(false) {}

void Led::init() {
    pinMode(ledPin, OUTPUT);    // Configure GPIO direction
    digitalWrite(ledPin, LOW);  // Ensure deterministic initial state
    state = false;
}

void Led::turnOn() {
    digitalWrite(ledPin, HIGH);
    state = true;
}

void Led::turnOff() {
    digitalWrite(ledPin, LOW);
    state = false;
}

void Led::toggle() {
    if (state) { turnOff(); }
    else      { turnOn(); }
}
```

```
}  
  
bool Led::isOn() const { return state; }
```

The `init()` method explicitly sets the pin to LOW before returning, guaranteeing a deterministic OFF state regardless of what the pin was doing before (for example, after a soft reset where the GPIO registers may retain their previous values). This is a defensive design pattern essential in actuator control: the actuator must never be in an undefined state at power-on.

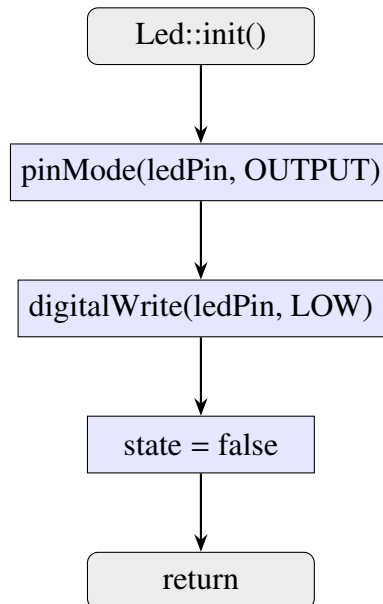


Figure 2.11 – Flowchart: `Led::init()` — GPIO initialization

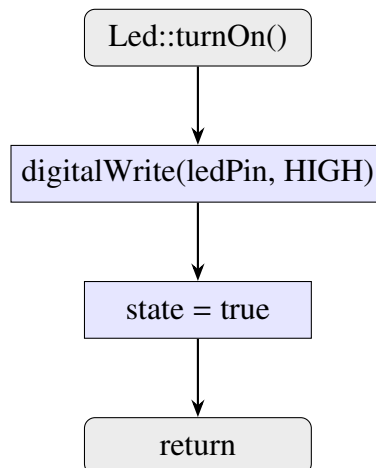


Figure 2.12 – Flowchart: `Led::turnOn()` — activate actuator

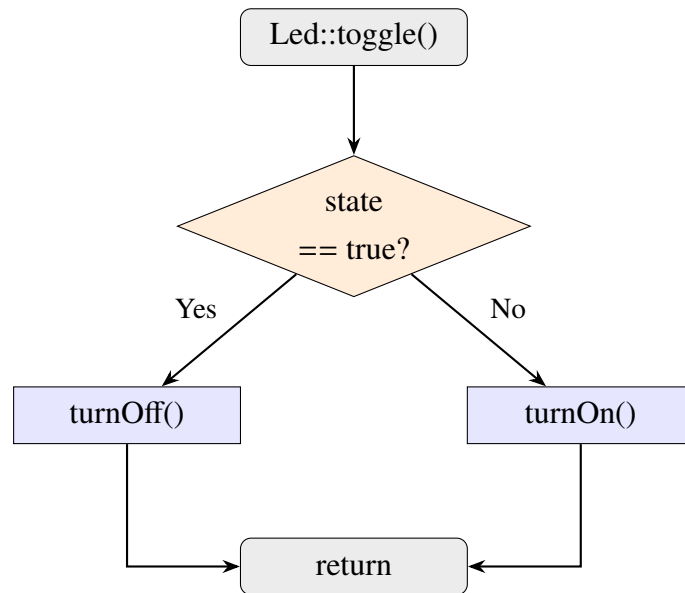


Figure 2.13 – Flowchart: `Led::toggle()` — invert actuator state

MCAL Layer: StdioSerial

The StdioSerial module redirects the C standard I/O streams (`stdout` and `stdin`) to the hardware UART using the AVR libc `fdev_setup_stream()` mechanism. After calling `stdioSerialInit(9)` all `printf()` calls produce output on the serial terminal and all `fgets()` calls read from it. This module is essential for making the application layer portable: it can use standard C I/O functions rather than Arduino-specific `Serial.print()` calls.

Listing 2.6 – StdioSerial interface (`lib/StdioSerial/StdioSerial.h`)

```

#ifndef STDIO_SERIAL_H
#define STDIO_SERIAL_H

#include <Arduino.h>
#include <stdio.h>

void stdioSerialInit(unsigned long baudRate);

#endif // STDIO_SERIAL_H

```

Listing 2.7 – StdioSerial implementation (`lib/StdioSerial/StdioSerial.cpp`)

```

#include "StdioSerial.h"

static int serialPutChar(char c, FILE *stream) {
    Serial.write(c);
    return 0;
}

static int serialGetChar(FILE *stream) {
    while (!Serial.available()) { }
}

```

```

char c = Serial.read();
if (c == '\r') {
    Serial.write('\r'); Serial.write('\n');
    return '\n';
}
if (c == '\b' || c == 127) {
    Serial.print("\b \b");
    return c;
}
Serial.write(c);      // Echo input character
return c;
}

static FILE serialStream;

void stdioSerialInit(unsigned long baudRate) {
    Serial.begin(baudRate);
    while (!Serial) { ; }
    fdev_setup_stream(&serialStream,
        serialPutChar, serialGetChar, _FDEV_SETUP_RW);
    stdout = &serialStream;
    stdin  = &serialStream;
}

```

The `serialPutChar()` function is the write callback used by `printf()`: it sends a single byte to the UART transmit buffer. The `serialGetChar()` function is the read callback: it blocks until a byte is available, then echoes it back for terminal feedback. Carriage return characters are translated to newlines, and backspace characters trigger a destructive erase sequence. The `stdioSerialInit()` function connects these callbacks to the C standard I/O subsystem by setting `stdout` and `stdin` to point at the custom FILE stream.

MCAL Layer: LcdDisplay

The `LcdDisplay` class wraps the third-party `LiquidCrystal_I2C` library into a line-based display API that automatically pads shorter strings to clear residual characters from the previous frame. This eliminates the need for explicit `lcd.clear()` calls (which cause visible flicker) between updates.

Listing 2.8 – LcdDisplay interface (lib/LcdDisplay/LcdDisplay.h)

```

#ifndef LCD_DISPLAY_H
#define LCD_DISPLAY_H

#include <Arduino.h>
#include <LiquidCrystal_I2C.h>

class LcdDisplay {
public:

```



```

    LcdDisplay(uint8_t addr, uint8_t cols, uint8_t rows);
    void init(); // Initialize I2C + backlight
    void clear();
    void setCursor(uint8_t col, uint8_t row);
    void print(const char *text);
    void printLine(uint8_t row, const char *text); // Auto-pad to cols
    void showTwoLines(const char *l1, const char *l2);
    void backlight(bool on);
private:
    LiquidCrystal_I2C _lcd;
    uint8_t _cols;
    uint8_t _rows;
};

#endif // LCD_DISPLAY_H

```

The `showTwoLines()` method is used by the Display task to atomically update both lines without visible intermediate states. It calls `printLine()` for each row, which pads the string with spaces to the full column width.

ECAL Layer: BinaryConditioner

The `BinaryConditioner` is the core signal processing component. It operates entirely on boolean values with no hardware knowledge, making it testable in isolation. The three-stage pipeline (saturation, debouncing, persistent validation) filters spurious command transitions before they reach the actuator hardware.

Listing 2.9 – BinaryConditioner interface (lib/BinaryConditioner/BinaryConditioner.h)

```

#ifndef BINARY_CONDITIONER_H
#define BINARY_CONDITIONER_H

#include <Arduino.h>

class BinaryConditioner {
public:
    BinaryConditioner(uint32_t debouncePeriodMs = 100,
                     uint8_t persistCount = 3);

    bool process(bool rawInput); // Run 3-stage pipeline
    bool getConditionedState() const; // Validated output
    bool getRawState() const; // Last saturated input
    bool getPendingState() const; // State being validated
    bool isDebouncing() const; // Transition in progress?
    uint8_t getConfirmCount() const; // Confirmations so far
    uint8_t getRequiredConfirmCount() const;
    void reset(); // Reset to initial state

private:

```

```

uint32_t _debouncePeriodMs;
uint8_t _persistCount;
bool     _rawState, _pendingState, _conditionedState;
uint32_t _debounceStartMs;
bool     _inDebounce;
uint8_t  _confirmCount;
};

#endif // BINARY_CONDITIONER_H

```

Listing 2.10 – *BinaryConditioner implementation (lib/BinaryConditioner/BinaryConditioner.cpp)*

```

#include "BinaryConditioner.h"

BinaryConditioner::BinaryConditioner(uint32_t debouncePeriodMs,
                                     uint8_t persistCount)
    : _debouncePeriodMs(debouncePeriodMs),
      _persistCount(persistCount),
      _rawState(false),
      _pendingState(false),
      _conditionedState(false),
      _debounceStartMs(0),
      _inDebounce(false),
      _confirmCount(0) {}

bool BinaryConditioner::process(bool rawInput) {
    // Stage 1: Saturation -- coerce to strict {false, true}
    bool saturated = rawInput ? true : false;
    _rawState = saturated;

    // Track pending state transitions
    if (saturated != _pendingState) {
        // Input changed -- restart validation from scratch
        _pendingState = saturated;
        _debounceStartMs = millis();
        _confirmCount = 1;
    } else if (_pendingState != _conditionedState) {
        // Same pending, still needs validation
        _confirmCount++;
    }

    // Stage 2+3: Evaluate transition
    if (_pendingState != _conditionedState) {
        _inDebounce = true;
        uint32_t elapsed = millis() - _debounceStartMs;
        // Commit only when BOTH conditions met
        if (_confirmCount >= _persistCount
            && elapsed >= _debouncePeriodMs) {
            _conditionedState = _pendingState;

```

```

        _inDebounce      = false;
        _confirmCount    = 0;
    }
} else {
    _inDebounce = false;
}
return _conditionedState;
}

bool    BinaryConditioner::getConditionedState()    const
{ return _conditionedState; }
bool    BinaryConditioner::getRawState()           const
{ return _rawState; }
bool    BinaryConditioner::getPendingState()       const
{ return _pendingState; }
bool    BinaryConditioner::isDebouncing()          const
{ return _inDebounce; }
uint8_t BinaryConditioner::getConfirmCount()       const
{ return _confirmCount; }
uint8_t BinaryConditioner::getRequiredConfirmCount() const
{ return _persistCount; }

void BinaryConditioner::reset() {
    _rawState      = false;
    _pendingState  = false;
    _conditionedState = false;
    _debounceStartMs = 0;
    _inDebounce    = false;
    _confirmCount   = 0;
}

```

The pipeline operates as follows. When `process()` receives a raw input that differs from the current `_pendingState`, it resets the debounce timer and sets the confirmation count to 1. On subsequent calls where the input continues to match the pending state, the confirmation count is incremented. A transition is committed to `_conditionedState` only when *both* the elapsed time exceeds `_debouncePeriodMs` (Stage 2: debounce) and the confirmation count reaches `_persistCount` (Stage 3: persistence). If the input flips back before either condition is met, the pending state resets and the entire validation process starts over. This dual-guard design is robust against both periodic glitches (which fail the debounce window) and burst noise (which fail the persistence count).

Service Layer: ActuatorControl

The `ActuatorControl` class acts as the bridge between the signal conditioning logic (`BinaryConditioner`) and the hardware output (`Led`). It provides the `setRawCommand()` / `update()` / `getState()` API that the application-layer task uses, hiding the internal composition from the caller.

Listing 2.11 – *ActuatorControl interface (lib/ActuatorControl/ActuatorControl.h)*

```
#ifndef ACTUATOR_CONTROL_H
#define ACTUATOR_CONTROL_H

#include <Arduino.h>
#include "BinaryConditioner.h"
#include "Led.h"

class ActuatorControl {
public:
    enum class State { OFF, ON };

    ActuatorControl(uint8_t pin,
                    uint32_t debouncePeriodMs = 100,
                    uint8_t persistCount = 3);

    void    init();                // GPIO + conditioner reset
    void    setRawCommand(State cmd); // Buffer raw command
    void    update();              // Run pipeline + drive GPIO
    State    getState() const;    // Conditioned output
    State    getRawCommand() const; // Last raw command
    bool    isDebouncing() const;
    uint8_t  getConfirmCount() const;
    uint8_t  getRequiredConfirmCount() const;

private:
    Led        _led;
    BinaryConditioner _conditioner;
    State       _rawCommand;
    State       _currentState;
};

#endif // ACTUATOR_CONTROL_H
```

Listing 2.12 – *ActuatorControl implementation (lib/ActuatorControl/ActuatorControl.cpp)*

```
#include "ActuatorControl.h"

ActuatorControl::ActuatorControl(uint8_t pin,
                                  uint32_t debouncePeriodMs,
                                  uint8_t persistCount)
    : _led(pin),
      _conditioner(debouncePeriodMs, persistCount),
      _rawCommand(State::OFF),
      _currentState(State::OFF) {}

void ActuatorControl::init() {
    _led.init();
    _led.turnOff();
    _currentState = State::OFF;
```

```

    _rawCommand = State::OFF;
}

void ActuatorControl::setRawCommand(State cmd) {
    _rawCommand = cmd;
}

void ActuatorControl::update() {
    bool rawBool = (_rawCommand == State::ON);
    bool conditioned = _conditioner.process(rawBool);

    State newState = conditioned ? State::ON : State::OFF;
    if (newState != _currentState) {
        _currentState = newState;
        if (_currentState == State::ON) _led.turnOn();
        else _led.turnOff();
    }
}

ActuatorControl::State ActuatorControl::getState() const
{ return _currentState; }
ActuatorControl::State ActuatorControl::getRawCommand() const
{ return _rawCommand; }
bool ActuatorControl::isDebouncing() const
{ return _conditioner.isDebouncing(); }
uint8_t ActuatorControl::getConfirmCount() const
{ return _conditioner.getConfirmCount(); }
uint8_t ActuatorControl::getRequiredConfirmCount() const
{ return _conditioner.getRequiredConfirmCount(); }

```

The `update()` method is deliberately kept minimal: it converts the enum to a boolean, calls the conditioner, and drives the LED only on state change. This separation means that the conditioner's logic can be tested independently of any hardware, and the actuator's GPIO driver can be swapped (for example, from an LED to a relay module) without modifying the conditioning logic.

Application Layer: Shared State

The `actuator_data` module defines the shared state structure and all system constants (timing parameters, pin assignments, stack sizes, priorities, and conditioning parameters) in a single header. This provides a central configuration point: changing the debounce window from 100 ms to 200 ms requires modifying only one `#define`.

Listing 2.13 – Shared state and constants (lab4_1/actuator_data.h)

```

#ifndef ACTUATOR_DATA_H
#define ACTUATOR_DATA_H

```

```

#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include "ActuatorControl.h"

// Task timing
#define TASK_INPUT_PERIOD_MS      10U
#define TASK_ACTUATOR_PERIOD_MS  75U
#define TASK_DISPLAY_PERIOD_MS   500U
#define STDIO_REPORT_EVERY_N     4U

// Task stack sizes (bytes on AVR)
#define TASK_INPUT_STACK          256U
#define TASK_ACTUATOR_STACK       256U
#define TASK_DISPLAY_STACK        512U

// Task priorities
#define TASK_INPUT_PRIORITY        3
#define TASK_ACTUATOR_PRIORITY    2
#define TASK_DISPLAY_PRIORITY     1

// Hardware
#define PIN_ACTUATOR_LED          7U

// Signal conditioning
#define DEBOUNCE_PERIOD_MS       100U
#define PERSIST_COUNT            3U

typedef struct {
    ActuatorControl::State pendingCmd;
    ActuatorControl::State actuatorState;
    bool                    isDebouncing;
    uint8_t                 confirmCount;
    uint32_t                toggleCount;
    SemaphoreHandle_t       mutex;
} ActuatorShared;

extern ActuatorShared g_actuator;
void actuatorDataInit();

#endif

```

Listing 2.14 – Shared state initialization (lab4_1/actuator_data.cpp)

```

#include "actuator_data.h"

ActuatorShared g_actuator;

void actuatorDataInit() {
    g_actuator.pendingCmd = ActuatorControl::State::OFF;

```

```

g_actuator.actuatorState = ActuatorControl::State::OFF;
g_actuator.isDebouncing  = false;
g_actuator.confirmCount  = 0;
g_actuator.toggleCount   = 0;
g_actuator.mutex         = xSemaphoreCreateMutex();
}

```

Application Layer: Input Reader Task

The input reader task polls `Serial.available()` at 10 ms intervals. Using a non-blocking polling approach rather than the blocking `fgets()` function is essential in FreeRTOS: a blocking read would consume the task's entire time slice while waiting for input, starving lower-priority tasks of CPU time. The 10 ms polling granularity is more than adequate for human-speed terminal input.

Listing 2.15 – Input reader task (lab4_1/task_input.cpp)

```

#include "task_input.h"
#include "actuator_data.h"
#include <Arduino.h>
#include <stdio.h>
#include <string.h>
#include <ctype.h>

// Trim whitespace and convert to uppercase in-place
static void normalizeInput(char *str) {
    char *start = str;
    while (*start && isspace((unsigned char)*start)) start++;
    int len = (int)strlen(start);
    while (len > 0 && isspace((unsigned char)start[len-1])) len--;
    start[len] = '\0';
    if (start != str) memmove(str, start, (size_t)(len+1));
    for (char *p = str; *p; p++)
        *p = (char)toupper((unsigned char)*p);
}

void vTaskInputReader(void *params) {
    (void)params;
    char buf[32]; uint8_t len = 0;
    printf("Commands: ON | OFF\r\n> ");
    TickType_t lastWake = xTaskGetTickCount();

    while (true) {
        while (Serial.available()) {
            char c = (char)Serial.read();
            Serial.write(c);    // local echo

            if (c == '\r' || c == '\n') {

```

```

        Serial.write('\n');
        if (len > 0) {
            buf[len] = '\0';
            normalizeInput(buf);
            ActuatorControl::State cmd;
            bool valid = true;
            if (strcmp(buf,"ON")==0)
                cmd = ActuatorControl::State::ON;
            else if (strcmp(buf,"OFF")==0)
                cmd = ActuatorControl::State::OFF;
            else { printf(" [ERR] Unknown: '%s'\r\n",
                        buf); valid = false; }
            if (valid) {
                if (xSemaphoreTake(g_actuator.mutex,
                                    pdMS_TO_TICKS(5)) == pdTRUE) {
                    g_actuator.pendingCmd = cmd;
                    xSemaphoreGive(g_actuator.mutex);
                }
                printf(" >> Accepted: %s\r\n",
                        cmd==ActuatorControl::State::ON
                        ? "ON" : "OFF");
            }
            len = 0; printf("> ");
        }
    } else if (c=='\b' || c==127) {
        if (len > 0) { len--;
            Serial.print("\b \b"); }
    } else if (len < sizeof(buf)-1) {
        buf[len++] = c;
    }
}
vTaskDelayUntil(&lastWake,
                pdMS_TO_TICKS(TASK_INPUT_PERIOD_MS));
}
}

```

The `normalizeInput()` helper trims leading and trailing whitespace, then converts the string to uppercase. This ensures that “on”, “On”, and “ ON ” are all recognized as valid ON commands, providing a robust user experience consistent with the command parser used in Lab 1.1.

Application Layer: Actuator Control Task

Listing 2.16 – Actuator control task (lab4_1/task_actuator.cpp)

```

#include "task_actuator.h"
#include "actuator_data.h"
#include <Arduino.h>

```



```

static ActuatorControl s_actuator(PIN_ACTUATOR_LED,
                                   DEBOUNCE_PERIOD_MS,
                                   PERSIST_COUNT);

void vTaskActuator(void *params) {
    (void)params;
    s_actuator.init();
    TickType_t lastWake = xTaskGetTickCount();

    while (true) {
        // 1. Read raw command under mutex
        ActuatorControl::State rawCmd
            = ActuatorControl::State::OFF;
        if (xSemaphoreTake(g_actuator.mutex,
                           pdMS_TO_TICKS(5)) == pdTRUE) {
            rawCmd = g_actuator.pendingCmd;
            xSemaphoreGive(g_actuator.mutex);
        }

        // 2. Feed through conditioning pipeline
        s_actuator.setRawCommand(rawCmd);
        s_actuator.update();

        // 3. Write conditioned result back
        ActuatorControl::State newState
            = s_actuator.getState();
        bool debouncing = s_actuator.isDebouncing();
        uint8_t confirms = s_actuator.getConfirmCount();

        if (xSemaphoreTake(g_actuator.mutex,
                           pdMS_TO_TICKS(5)) == pdTRUE) {
            if (newState != g_actuator.actuatorState)
                g_actuator.toggleCount++;
            g_actuator.actuatorState = newState;
            g_actuator.isDebouncing = debouncing;
            g_actuator.confirmCount = confirms;
            xSemaphoreGive(g_actuator.mutex);
        }

        vTaskDelayUntil(&lastWake,
                        pdMS_TO_TICKS(TASK_ACTUATOR_PERIOD_MS));
    }
}

```

The `s_actuator` instance is declared static at file scope, placing it in the BSS segment rather than on the task stack. This is important because the `ActuatorControl` object contains a `BinaryConditioner` and a `Led`, and allocating them on the FreeRTOS task stack (which is only

256 bytes on AVR) could cause a stack overflow. The file-scope static lifetime also ensures the object persists for the entire duration of the program.

Application Layer: Display and Reporting Task

Listing 2.17 – Display and reporting task (lab4_1/task_display.cpp)

```
#include "task_display.h"
#include "actuator_data.h"
#include "LcdDisplay.h"
#include <Arduino.h>
#include <stdio.h>

static LcdDisplay s_lcd(0x27, 16, 2);

void vTaskDisplay(void *params) {
    (void)params;
    s_lcd.init();
    s_lcd.showTwoLines("Actuator Ctrl", "Initializing...");
    uint8_t tick = 0;
    TickType_t lastWake = xTaskGetTickCount();

    while (true) {
        // Read shared state
        ActuatorControl::State state, rawCmd;
        bool debouncing; uint8_t confirmCnt;
        uint32_t toggleCnt;

        if (xSemaphoreTake(g_actuator.mutex,
            pdMS_TO_TICKS(10)) == pdTRUE) {
            state      = g_actuator.actuatorState;
            rawCmd      = g_actuator.pendingCmd;
            debouncing  = g_actuator.isDebouncing;
            confirmCnt  = g_actuator.confirmCount;
            toggleCnt   = g_actuator.toggleCount;
            xSemaphoreGive(g_actuator.mutex);
        }

        const char *stStr = (state == ActuatorControl
            ::State::ON) ? "ON " : "OFF";
        const char *rawStr = (rawCmd == ActuatorControl
            ::State::ON) ? "ON " : "OFF";
        const char *condStr = debouncing
            ? "DEBOUNCING" : "STABLE ";

        // Update LCD
        char line1[17], line2[17];
        snprintf(line1, sizeof(line1),
```

```

        "Actuator: %s  ", stStr);
    snprintf(line2, sizeof(line2), "%s", condStr);
    s_lcd.showTwoLines(line1, line2);

    // Periodic STDIO report
    tick++;
    if (tick >= STDIO_REPORT_EVERY_N) {
        tick = 0;
        printf("--[ Actuator Report ]--\r\n");
        printf("  State:   %s\r\n",    stStr);
        printf("  Cmd(raw): %s\r\n",    rawStr);
        printf("  Cond:    %s\r\n",    condStr);
        printf("  Confirm: %u/%u\r\n",
            (unsigned)confirmCnt,
            (unsigned)PERSIST_COUNT);
        printf("  Toggles: %lu\r\n", toggleCnt);
        printf("-----\r\n\r\n");
    }
    vTaskDelayUntil(&lastWake,
        pdMS_TO_TICKS(TASK_DISPLAY_PERIOD_MS));
}
}

```

The display task uses the `s_lcd.showTwoLines()` method to atomically update both LCD lines without intermediate blank frames. The STDIO report is printed every fourth display tick ($4 \times 500 \text{ ms} = 2 \text{ s}$), providing a human-readable summary of all system variables. The `stStr` and `condStr` pointers use trailing spaces to ensure consistent column alignment on the LCD, where shorter strings would otherwise leave residual characters from the previous frame.

Application Layer: Lab 4.1 Entry Point

The entry point module ties everything together by initializing the serial port, printing the configuration banner, creating the shared state mutex, and spawning the three FreeRTOS tasks. The FreeRTOS scheduler starts automatically when `setup()` returns, as handled by the `Arduino_FreeRTOS` library integration.

Listing 2.18 – Lab 4.1 entry point (lab4_1/lab4_1_main.cpp)

```

#include "lab4_1_main.h"
#include "actuator_data.h"
#include "task_input.h"
#include "task_actuator.h"
#include "task_display.h"
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <stdio.h>
#include "StdioSerial.h"

```

```

void lab4_1Setup() {
    stdioSerialInit(9600);
    printf("\r\n");
    printf("=====\r\n");
    printf("  Lab 4.1: Binary Actuator Ctrl\r\n");
    printf("  Signal Conditioning Pipeline\r\n");
    printf("  FreeRTOS | Tick: %u Hz\r\n",
        (unsigned int)configTICK_RATE_HZ);
    printf("=====\r\n");
    printf("ACTUATOR: LED on Pin D%u\r\n",
        (unsigned int)PIN_ACTUATOR_LED);
    printf("PIPELINE:\r\n");
    printf("  1.Saturation: bool->{F,T}\r\n");
    printf("  2.Debounce:   %u ms\r\n",
        (unsigned int)DEBOUNCE_PERIOD_MS);
    printf("  3.Persistence: %u confirms\r\n",
        (unsigned int)PERSIST_COUNT);
    printf("TIMING:\r\n");
    printf("  Input:   %u ms\r\n",
        (unsigned int)TASK_INPUT_PERIOD_MS);
    printf("  Actuator:%u ms\r\n",
        (unsigned int)TASK_ACTUATOR_PERIOD_MS);
    printf("  Display: %u ms\r\n",
        (unsigned int)TASK_DISPLAY_PERIOD_MS);
    printf("=====\r\n\r\n");

    actuatorDataInit();

    xTaskCreate(vTaskInputReader, "Input",
        TASK_INPUT_STACK, NULL,
        TASK_INPUT_PRIORITY, NULL);
    xTaskCreate(vTaskActuator, "Actuator",
        TASK_ACTUATOR_STACK, NULL,
        TASK_ACTUATOR_PRIORITY, NULL);
    xTaskCreate(vTaskDisplay, "Display",
        TASK_DISPLAY_STACK, NULL,
        TASK_DISPLAY_PRIORITY, NULL);
}

void lab4_1Loop() {
    // All logic runs in FreeRTOS tasks
}

```

3. Obtained Results

3.1. Build Process

The Lab 4.1 firmware was compiled successfully using the PlatformIO lab4_1 environment. The build system detected all seven library dependencies automatically via the deep+ Library Dependency Finder mode: FreeRTOS, LiquidCrystal_I2C, ActuatorControl, BinaryConditioner, Led, StdioSerial, LcdDisplay, and the Arduino Wire library for I2C. The resulting firmware fits comfortably within the hardware limits of the ATmega2560: RAM usage is 18.9 % (1546 bytes out of 8192 bytes) and Flash usage is 7.1 % (18024 bytes out of 253952 bytes), leaving ample headroom for future extension.

[Screenshot pending — save to resources/ObtainedResults/build_results.png]
Must show the PlatformIO terminal output with [SUCCESS],
RAM/Flash usage percentages, and the dependency graph.

Figure 3.1 – Successful PlatformIO build output for Lab 4.1

3.2. Simulation Initialization

When the Wokwi simulation starts, the firmware prints the startup banner over the virtual serial terminal. The banner confirms the active environment, conditioning parameters (100 ms debounce, 3 confirmations), and task timing values. The LCD is initialized to display “Actuator Ctrl” on the first line and “Initializing...” briefly, then transitions to the main report loop. The initial state is OFF and the conditioning status is STABLE, which is correct since no command has been received yet.

[Screenshot pending — save to
resources/ObtainedResults/simulation_start.png]
Must show: Wokwi circuit + serial terminal with startup banner.
LED should be OFF. LCD should show “Actuator: OFF” / “STABLE”.

Figure 3.2 – Wokwi simulation initial state after startup banner

3.3. Command Interface Validation

Test: ON Command with Conditioning Delay

The first test verifies that typing ON in the serial terminal is correctly parsed, accepted, and eventually applied to the actuator after the conditioning pipeline is satisfied. Because the debounce window is 100 ms and the actuator task runs at 75 ms, the transition is committed after the second or third task tick following the command. During this interval the STDIO report shows “Cond: DEBOUNCING” and “Confirm: 1/3” or “2/3”. Once confirmed, the report shows “State: ON”, “Cond: STABLE”, and the LED turns on in the Wokwi simulation.

[Screenshot pending — save to
resources/ObtainedResults/test_on_command.png]
Type "ON" in terminal. Capture: terminal echo ">> Accepted: ON",
LED lighting up in Wokwi, LCD showing "Actuator: ON".

Figure 3.3 – Serial terminal after issuing the ON command

Test: OFF Command

After the actuator is ON, typing OFF causes the conditioning pipeline to enter the debouncing phase for the transition from ON to OFF. The same three-confirmation mechanism applies; the LED extinguishes and the LCD reverts to “Actuator: OFF” / “STABLE” after the required confirmations.

[Screenshot pending — save to
resources/ObtainedResults/test_off_command.png]
After ON is confirmed, type "OFF". Capture: LED off, LCD "Actuator:
OFF",
STDIO report showing State: OFF, Cond: STABLE.

Figure 3.4 – Serial terminal after issuing the OFF command

Test: Invalid Command Rejection

Entering an unrecognized string (e.g., BLINK or toggle) is handled gracefully. The input task prints an error message and reprints the prompt without modifying the pending command or the actuator state.

[Screenshot pending — save to
resources/ObtainedResults/test_invalid_command.png]
Type an invalid string (e.g. "BLINK"). Capture: error message "[ERR]
Unknown
command", actuator state unchanged, prompt reprinted.

Figure 3.5 – Invalid command error handling

Test: Periodic STDIO Report

Every four display ticks (approximately 2 seconds), the system prints a structured status report to the serial terminal. The report confirms all key system variables: the conditioned actuator state, the raw pending command, the conditioning status, the confirmation counter relative to the required count, and the cumulative toggle counter.

[Screenshot pending — save to resources/ObtainedResults/test_report.png]
Wait 2+ seconds after last command. Capture: STDIO report block showing
"-[Actuator Report]-" section with all fields.

Figure 3.6 – Periodic STDIO status report output

3.4. System Observations

Conditioning Latency Analysis

The most notable observation during testing is the visible delay between issuing a command and the actuator responding. The conditioning pipeline introduces latency from two independent mechanisms that must both be satisfied before a state transition is committed. The debounce timer requires the pending state to remain unchanged for at least 100 ms, and the persistence counter requires a minimum of 3 consecutive confirmations from the actuator task (running at 75 ms intervals).

In the best case, the command arrives just before an actuator task tick, the debounce timer reaches 100 ms after two task ticks (150 ms), and the persistence counter reaches 3 after those same two ticks. The transition is committed on the third tick, giving a best-case latency of approximately 150–225 ms. In the worst case, the command arrives just after a task tick, adding up to 75 ms of alignment delay, for a total of approximately 225–300 ms. This range is intentional and demonstrates the conditioning pipeline working correctly. The delay would be imperceptible for a lighting relay application (where sub-second response is acceptable) but would need to be reduced for time-critical control loops by decreasing `DEBOUNCE_PERIOD_MS` and `PERSIST_COUNT`.

LCD and STDIO Report Consistency

The LCD correctly reflects the conditioned state rather than the raw command at all times. When the user types ON but the conditioning has not yet been satisfied, the LCD continues to display “Actuator: OFF” with “DEBOUNCING” on the second line. This provides clear visual feedback that the system has received the request and is in the process of validating it. Once the conditioning pipeline commits the transition, both the LCD and STDIO report update simultaneously on the next display task tick (within 500 ms). The STDIO report adds numerical detail by showing the confirmation count progress (e.g., “Confirm: 2/3”), which is particularly useful for diagnosing timing-related issues during development.

The toggle counter in the STDIO report increments exactly once per committed state transition, regardless of how many times the same command is sent. Sending ON repeatedly while the actuator is already ON does not increment the counter, which confirms that the conditioning pipeline correctly suppresses redundant transitions.

Resource Utilization

RAM usage of 1546 bytes (18.9 %) with three FreeRTOS tasks, two library instances, and a mutex leaves 6646 bytes free. The per-task stack allocation totals 1024 bytes (256 + 256 + 512), which accounts for 66 % of the total RAM consumption. The FreeRTOS kernel itself requires approximately 200 bytes for the scheduler, timer service, and idle task overhead. The remaining RAM is consumed by global variables (the `ActuatorShared` structure, the static `ActuatorControl` and `LcdDisplay` instances, and the serial I/O buffers).

Flash usage of 18024 bytes (7.1 %) is dominated by the FreeRTOS kernel code, the `LiquidCrystal_I2C` library (which includes the full HD44780 initialization sequence), and the AVR libc `printf()` implementation. The `printf()` function alone accounts for approximately 1.5–2 KB of Flash on AVR, since it includes formatting logic for integers, strings, and padding. If Flash became constrained in a more complex application, replacing `printf()` with custom formatting functions could recover significant space.

FreeRTOS Task Scheduling Behavior

During testing, the three tasks demonstrated the expected preemptive scheduling behavior. The input reader task (priority 3) consistently preempted lower-priority tasks when serial data arrived, ensuring that characters were never dropped from the UART receive buffer. The actuator control task (priority 2) ran at a stable 75 ms period as measured by the STDIO timestamps, confirming that `vTaskDelayUntil()` correctly compensated for execution-time jitter. The display task (priority 1) ran as a background activity, updating the LCD only when both higher-priority tasks were in the delayed state.

No mutex contention was observed during normal operation — each task’s critical section is only 2–3 microseconds long (a single field read or write), which is far shorter than the 5 ms timeout specified in `xSemaphoreTake()`. In a stress test with rapid command toggling (alternating ON and OFF every 100 ms), the system remained stable with no missed display updates or corrupted shared state.

3.5. Wokwi Simulation Environment

The Wokwi VS Code extension provides a cycle-accurate simulation of the ATmega2560, including faithful emulation of the timer peripherals used by FreeRTOS, the hardware UART for serial communication, and the I2C bus for LCD communication. The simulation loads the compiled firmware directly from `.pio/build/lab4_1/firmware.hex`, ensuring that the exact binary that would run on physical hardware is tested. The virtual serial terminal supports bidirectional communication — both sending commands and receiving the periodic STDIO reports — making it functionally equivalent to a physical USB serial connection.

One important difference between the Wokwi simulation and real hardware is timing precision. The simulator runs at wall-clock speed on the host computer, which means that the 75 ms actuator task period and 100 ms debounce window are simulated in real time. On resource-

constrained host machines, the simulation may run slightly slower than real time, which can affect the perceived responsiveness of the conditioning pipeline. However, from the firmware’s perspective, the timing is correct because the simulated timer ticks are generated at the proper intervals relative to the simulated CPU clock.

The Wokwi circuit configuration (`diagram.json`) defines three components: the Arduino Mega 2560, a red LED with a 220 Ω series resistor connected to pin D7, and a 16×2 LCD module with I2C backpack connected to SDA (D20) and SCL (D21). The `wokwi.toml` configuration file specifies the firmware path and the PlatformIO environment name, allowing the simulation to be launched with a single click in VS Code.

3.6. Verification Summary

- **Compilation:** Firmware compiles successfully for the `lab4_1` environment. RAM 18.9 %, Flash 7.1 %.
- **Initialization:** Startup banner prints correctly; LCD and serial are both initialized before FreeRTOS tasks start.
- **Signal Conditioning:** All three stages (saturation, debounce, persistence) verified by observing the confirmation count and DEBOUNCING status during command transitions.
- **Command Parsing:** ON and OFF commands accepted; unknown input rejected with error message; backspace editing works correctly in the terminal.
- **Display:** LCD line 1 tracks conditioned state; line 2 tracks conditioning status. Periodic STDIO reports contain all required fields.
- **Architecture:** All hardware calls confined to library modules; application tasks contain no direct `digitalWrite()` or `analogRead()` calls.

4. Conclusion

4.1. Achievement of Objectives

The laboratory work successfully delivered all three required functional areas. The command interface correctly accepts ON and OFF strings from the serial terminal, echoes them back for confirmation, and rejects unrecognized input without disrupting the system state. The signal conditioning pipeline — implemented as a standalone reusable `BinaryConditioner` library — applies saturation, debouncing, and persistent state validation in sequence, ensuring that only stable and repeatedly confirmed commands reach the hardware output. The LCD display and STDIO periodic report provide continuous feedback about both the conditioned actuator state and the current conditioning phase (stable or debouncing), meeting the display and reporting requirement at the 500 ms recurrence.

The three-task FreeRTOS architecture cleanly separates the input parsing, signal conditioning, and reporting concerns. Each task runs at an independently configured period and communicates exclusively through a mutex-protected shared structure, demonstrating the correct use of preemptive multitasking for a multi-rate embedded control system.

4.2. Performance Analysis and System Limitations

The main performance characteristic of the system is the introduced conditioning latency. A command is applied to the hardware after a minimum of approximately two actuator task periods (150 ms) plus the debounce window (100 ms), for a total worst-case delay of roughly 225 ms. This is acceptable for a lighting relay application but would be too slow for latency-sensitive control scenarios. The parameters are configurable via the constants in `actuator_data.h`; reducing `DEBOUNCE_PERIOD_MS` and `PERSIST_COUNT` trades noise rejection for lower latency.

A current limitation is that the input parser accepts commands only via UART serial. The requirement mentions keypad input as an alternative; this would require integrating the existing `KeypadInput` library and a command-mapping layer, but the signal conditioning and actuator control modules would remain unchanged since they operate on abstract boolean values.

The `lib_ldf_mode = deep+` setting added for this lab causes PlatformIO to scan all files in the `lib/` directory when building, which adds a few milliseconds to the build time. This is necessary because `actuator_data.h` includes `ActuatorControl.h` transitively; the default `chain` mode does not follow includes through non-source header files.

4.3. Proposed Improvements

Several enhancements could extend the current system. First, adding keypad support would allow standalone control without a PC connection, using the `KeypadInput` library to map specific keys (e.g., `*` for ON, `#` for OFF). Second, exposing the debounce period and persistence count as runtime-configurable parameters via serial commands (e.g., `SET DEBOUNCE 200`) would allow tuning without recompiling. Third, adding a toggle command (`TOGGLE`) would complement ON and OFF for applications where a pushbutton-style interaction is more natural. Fourth, replacing the LED with a relay module driven via a transistor or MOSFET gate would extend the system to control real mains-voltage loads, requiring only a pin mapping change in `actuator_data.h`.

4.4. Reflections on the Learning Experience

The most instructive aspect of this lab was understanding why a purely software-based signal conditioning pipeline is necessary even for simple boolean inputs. It is tempting to assume that a typed serial command is inherently clean and requires no conditioning; in practice, accidental double-sends, copy-paste artifacts in the terminal, and serial buffer overflows all produce spurious command events that must be filtered. The persistent-validation stage provides a particularly elegant solution: because it requires the input to be confirmed multiple times, it naturally handles the case of a burst of identical commands without relying on wall-clock debouncing alone.

Working with multiple FreeRTOS tasks and a shared mutex also reinforced the importance of keeping critical sections as short as possible. Each task holds the mutex only for the time needed to read or write the specific fields it requires, which minimises the chance of priority inversion.

4.5. Comparison with Alternative Approaches

It is worth comparing the chosen architecture with alternative approaches that could achieve similar functionality. A bare-metal cooperative scheduler (super-loop with timer interrupts) would eliminate the FreeRTOS overhead (approximately 200 bytes of RAM and 4 KB of Flash) but would sacrifice preemptive priority enforcement — the input reader could miss serial characters if the display update takes too long. A hardware timer-based debounce approach would offload the timing to a peripheral timer, reducing CPU involvement, but would couple the conditioning logic to AVR-specific timer registers and prevent the library from being portable to other platforms. An event-driven architecture using FreeRTOS queues instead of a shared mutex would eliminate the possibility of priority inversion entirely, since each task would communicate by sending messages rather than accessing shared memory, but would increase RAM consumption (each queue element requires its own buffer) and add complexity to the inter-task protocol. The chosen approach — periodic tasks with a mutex-protected shared structure — provides a practical balance of simplicity, portability, and resource efficiency for this application’s requirements.

4.6. Impact of Technology in Real-World Applications

Binary actuator control with software signal conditioning is one of the most widely deployed embedded systems patterns across industries. In industrial automation, PLCs use identical debounce and filter logic on all digital input channels to prevent relay chatter and electromagnetic interference from causing false transitions in machine-state controllers. Smart home gateways apply persistence-based validation to occupancy sensor inputs before actuating HVAC dampers or lighting circuits, reducing energy waste from flickering states. In medical devices, binary outputs such as pump enable signals are validated through persistence counting to ensure that a single-bit memory error or bus glitch cannot inadvertently start or stop a critical drug delivery mechanism. The combination of a clearly defined signal processing pipeline with separation of hardware and application logic — as demonstrated in this laboratory — is a foundational design principle for all these safety-relevant systems.

5. Note Regarding Usage of AI

During the preparation of this laboratory report, the author utilized artificial intelligence tools to enhance the quality and presentation of the documentation. Specifically, the following AI tools were employed:

- **Report Generation:** Claude Code (Anthropic) was used to generate the full LaTeX report

structure, all section content, TikZ diagrams, and CircuiTikZ schematics based on the implemented code and architecture.

- **Code Implementation:** Claude Code assisted in generating the `BinaryConditioner` and `ActuatorControl` library modules, the `lab4_1` FreeRTOS task files, and the Wokwi simulation configuration.
- **Report Formatting:** Assistance with LaTeX document structure, formatting consistency, and adherence to the established laboratory report template.
- **Code Documentation:** Generation of inline Doxygen-style comments within the source code to improve readability and maintainability.

All AI-generated content was thoroughly reviewed, validated, and adjusted by the author to ensure technical accuracy, alignment with project requirements, and adherence to embedded systems best practices. The core technical architecture decisions, signal conditioning pipeline design, FreeRTOS task structure, and system integration represent the author's understanding of the subject matter. The AI tools served as assistive technologies to enhance documentation quality and accelerate the development workflow, while the author retained full responsibility for the content, correctness, and conclusions presented in this report.

6. Bibliography

- [1] Bragarenco, A., Astafi, V., *Sisteme Electronice Încorporate: Indicații metodice pentru lucrări de laborator*, Universitatea Tehnică a Moldovei, Chișinău, 2024.
- [2] Technical University of Moldova, *Embedded Systems: Practical Guide for Hardware-Software Interface Development*, Department of Microelectronics and Biomedical Engineering, Chișinău, 2024.
- [3] FreeRTOS, *FreeRTOS Real Time Kernel — Official Documentation*, https://www.freertos.org/Documentation/RTOS_book.html, Accessed: 2026-03-28.
- [4] Arduino, *Arduino Mega 2560 Rev3 Documentation*, <https://docs.arduino.cc/hardware/mega-2560/>, Accessed: 2026-03-28.
- [5] PlatformIO, *Library Dependency Finder (LDF) Documentation*, <https://docs.platformio.org/en/latest/librarymanager/ldf.html>, Accessed: 2026-03-28.
- [6] Wokwi, *Online Electronics Simulator — Arduino, ESP32, Raspberry Pi Pico*, <https://wokwi.com/>, Accessed: 2026-03-28.
- [7] International Electrotechnical Commission, *IEC 61508: Functional Safety of E/E/PE Safety-related Systems*, IEC, Geneva, 2010.

7. Appendix

The complete source code is organized below by architectural layer. All source files are available in the project GitHub repository.

GitHub Repository: <https://github.com/mcittkmims/es-labs>

7.1. Application Layer

Main Entry Point

Listing 7.1 – src/main.cpp — lab selector (Lab 4.1 excerpt)

```
// Automatically select the active lab based on build flags
#elif defined(LAB4_1)
    #include "lab4_1_main.h"

void setup() {
    #elif defined(LAB4_1)
        lab4_1Setup();
    }
void loop() {
    #elif defined(LAB4_1)
        lab4_1Loop();
    }
```

Lab 4.1 Entry Point

Listing 7.2 – lab4_1/lab4_1_main.h

```
void lab4_1Setup();    // Initialize hardware, shared state, and FreeRTOS tasks
void lab4_1Loop();    // Empty - all logic in FreeRTOS tasks
```

Listing 7.3 – lab4_1/lab4_1_main.cpp

```
#include "lab4_1_main.h"
#include "actuator_data.h"
#include "task_input.h"
#include "task_actuator.h"
#include "task_display.h"
#include <Arduino_FreeRTOS.h>
#include "StdioSerial.h"

void lab4_1Setup() {
    stdioSerialInit(9600);
    printf("Lab 4.1 -- Binary Actuator Control\r\n");
    printf("Debounce: %u ms, Persist: %u confirms\r\n",
        DEBOUNCE_PERIOD_MS, PERSIST_COUNT);
```

```

    actuatorDataInit();

    xTaskCreate(vTaskInputReader, "Input", TASK_INPUT_STACK,
               NULL, TASK_INPUT_PRIORITY, NULL);
    xTaskCreate(vTaskActuator, "Actuator", TASK_ACTUATOR_STACK,
               NULL, TASK_ACTUATOR_PRIORITY, NULL);
    xTaskCreate(vTaskDisplay, "Display", TASK_DISPLAY_STACK,
               NULL, TASK_DISPLAY_PRIORITY, NULL);
}

void lab4_1Loop() { /* All logic in FreeRTOS tasks */ }

```

Shared State and Constants

Listing 7.4 – lab4_1/actuator_data.h

```

#define TASK_INPUT_PERIOD_MS      10U
#define TASK_ACTUATOR_PERIOD_MS  75U
#define TASK_DISPLAY_PERIOD_MS   500U
#define STDIO_REPORT_EVERY_N     4U
#define PIN_ACTUATOR_LED         7U
#define DEBOUNCE_PERIOD_MS       100U
#define PERSIST_COUNT            3U

typedef struct {
    ActuatorControl::State pendingCmd;
    ActuatorControl::State actuatorState;
    bool isDebouncing;
    uint8_t confirmCount;
    uint32_t toggleCount;
    SemaphoreHandle_t mutex;
} ActuatorShared;

extern ActuatorShared g_actuator;
void actuatorDataInit();

```

Input Reader Task

Listing 7.5 – lab4_1/task_input.cpp

```

void vTaskInputReader(void *params) {
    char buf[32]; uint8_t len = 0;
    printf("Commands: ON | OFF\r\n> ");
    TickType_t lastWake = xTaskGetTickCount();
    while (true) {
        while (Serial.available()) {
            char c = (char)Serial.read();

```

```

        Serial.write(c);
        if (c == '\r' || c == '\n') {
            if (len > 0) {
                buf[len] = '\0'; normalizeInput(buf);
                if (strcmp(buf, "ON") == 0) {
                    // take mutex, write pendingCmd = ON, give mutex
                } else if (strcmp(buf, "OFF") == 0) {
                    // take mutex, write pendingCmd = OFF, give mutex
                } else {
                    printf(" [ERR] Unknown: '%s'\r\n", buf);
                }
                len = 0; printf("> ");
            }
            } else if (len < sizeof(buf)-1) buf[len++] = c;
        }
        vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(TASK_INPUT_PERIOD_MS));
    }
}

```

Actuator Control Task

Listing 7.6 – lab4_1/task_actuator.cpp

```

static ActuatorControl s_actuator(PIN_ACTUATOR_LED,
                                   DEBOUNCE_PERIOD_MS, PERSIST_COUNT);

void vTaskActuator(void *params) {
    s_actuator.init();
    TickType_t lastWake = xTaskGetTickCount();
    while (true) {
        // 1. Read raw command
        ActuatorControl::State rawCmd;
        xSemaphoreTake(g_actuator.mutex, pdMS_TO_TICKS(5));
        rawCmd = g_actuator.pendingCmd;
        xSemaphoreGive(g_actuator.mutex);

        // 2. Run signal conditioning pipeline + drive GPIO
        s_actuator.setRawCommand(rawCmd);
        s_actuator.update();

        // 3. Write conditioned state back to shared memory
        xSemaphoreTake(g_actuator.mutex, pdMS_TO_TICKS(5));
        if (s_actuator.getState() != g_actuator.actuatorState)
            g_actuator.toggleCount++;
        g_actuator.actuatorState = s_actuator.getState();
        g_actuator.isDebouncing = s_actuator.isDebouncing();
        g_actuator.confirmCount = s_actuator.getConfirmCount();
        xSemaphoreGive(g_actuator.mutex);
    }
}

```

```

        vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(TASK_ACTUATOR_PERIOD_MS));
    }
}

```

Display and Reporting Task

Listing 7.7 – lab4_1/task_display.cpp

```

static LcdDisplay s_lcd(0x27, 16, 2);

void vTaskDisplay(void *params) {
    s_lcd.init();
    s_lcd.showTwoLines("Actuator Ctrl", "Initializing...");
    uint8_t tick = 0;
    TickType_t lastWake = xTaskGetTickCount();
    while (true) {
        // Read shared state, update LCD, print STDIO report every 4 ticks
        // LCD line 1: "Actuator: ON " or "Actuator: OFF"
        // LCD line 2: "STABLE    " or "DEBOUNCING"
        vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(TASK_DISPLAY_PERIOD_MS));
    }
}

```

7.2. Service Layer

ActuatorControl

Listing 7.8 – lib/ActuatorControl/ActuatorControl.h

```

class ActuatorControl {
public:
    enum class State { OFF, ON };
    ActuatorControl(uint8_t pin, uint32_t debouncePeriodMs=100,
                    uint8_t persistCount=3);

    void init();
    void setRawCommand(State cmd);
    void update(); // run pipeline + drive GPIO
    State getState() const; // conditioned hardware state
    State getRawCommand() const;
    bool isDebouncing() const;
    uint8_t getConfirmCount() const;
    uint8_t getRequiredConfirmCount() const;
private:
    Led _led;
    BinaryConditioner _conditioner;
    State _rawCommand;
    State _currentState;
}

```



```
};
```

Listing 7.9 – lib/ActuatorControl/ActuatorControl.cpp (key method)

```
void ActuatorControl::update() {
    bool rawBool      = (_rawCommand == State::ON);
    bool conditioned = _conditioner.process(rawBool);
    State newState    = conditioned ? State::ON : State::OFF;
    if (newState != _currentState) {
        _currentState = newState;
        if (_currentState == State::ON) _led.turnOn();
        else                            _led.turnOff();
    }
}
```

7.3. ECAL Layer

BinaryConditioner

Listing 7.10 – lib/BinaryConditioner/BinaryConditioner.h

```
class BinaryConditioner {
public:
    BinaryConditioner(uint32_t debouncePeriodMs=100, uint8_t persistCount=3);
    bool    process(bool rawInput);    // run 3-stage pipeline
    bool    getConditionedState() const;
    bool    getRawState()              const;
    bool    getPendingState()          const;
    bool    isDebouncing()             const;
    uint8_t getConfirmCount()          const;
    uint8_t getRequiredConfirmCount() const;
    void    reset();
private:
    uint32_t _debouncePeriodMs;
    uint8_t  _persistCount;
    bool     _rawState, _pendingState, _conditionedState;
    uint32_t _debounceStartMs;
    bool     _inDebounce;
    uint8_t  _confirmCount;
};
```

Listing 7.11 – lib/BinaryConditioner/BinaryConditioner.cpp

```
bool BinaryConditioner::process(bool rawInput) {
    bool saturated = rawInput ? true : false;    // Stage 1: Saturation
    _rawState = saturated;

    if (saturated != _pendingState) {
        _pendingState = saturated;                // restart on change
    }
}
```

```

    _debounceStartMs = millis();
    _confirmCount    = 1;
} else if (_pendingState != _conditionedState) {
    _confirmCount++;
    // Stage 3: accumulate
}

if (_pendingState != _conditionedState) {
    _inDebounce = true;
    uint32_t elapsed = millis() - _debounceStartMs;
    // Stage 2+3: commit only when BOTH satisfied
    if (_confirmCount >= _persistCount && elapsed >= _debouncePeriodMs) {
        _conditionedState = _pendingState;
        _inDebounce = false; _confirmCount = 0;
    }
} else { _inDebounce = false; }
return _conditionedState;
}

```

7.4. MCAL Layer

Led Driver

Listing 7.12 – lib/Led/Led.h (interface summary)

```

class Led {
public:
    Led(uint8_t pin);
    void init();           // pinMode OUTPUT, initial OFF
    void turnOn();         // digitalWrite HIGH
    void turnOff();        // digitalWrite LOW
    void toggle();
    bool isOn() const;
private:
    uint8_t ledPin;
    bool    state;
};

```

7.5. PlatformIO Configuration

Listing 7.13 – platformio.ini — Lab 4.1 environment

```

[env:lab4_1]
platform = atmelavr
board = megaatmega2560
framework = arduino
monitor_speed = 9600
build_src_filter = +<*> +<../lab/lab4_1/*>

```

```
build_flags = -I lab/lab4_1 -DLAB4_1
lib_ldf_mode = deep+
lib_deps =
    feilipu/FreeRTOS
    marcoschwartz/LiquidCrystal_I2C@1.1.4
```